ComPDFKit

PDF SDK for Java

Developer Guide

Version 3.2.0

2026-10-09

Contents

1 Overview 6

1.1 ComPDFKit PDF SDK . . . 6

1.2 Key Features . . . 7

2 Get Started 9

2.1 Requirements . . . 9

2.1.1 Get ComPDFKit License Key . . . 9

2.1.2 System Requirements . . . 9

2.2 Java Package Structure . . . 9

2.3 Apply the license Key . . . 10

2.3.1 Copying the License Key . . . 10

2.3.2 Apply the License Key . . . 10

2.4 How to Run a Demo . . . 11

2.5 How to Use ComPDFKit PDF SDK for Java in Java Applications . . . 11

2.5.1 Create a New Project . . . 11

2.5.2 Add the Package of ComPDFKit PDF SDK for Java . . . 12

2.5.3 Troubleshooting . . . 14

2.6 Samples . . . 14

3 Guides 16

3.1 Basic Operations . . . 16

3.1.1 Overview . . . 16

3.1.2 Open a Document . . . 16

3.1.3 Save a Document . . . 17

3.1.4 Document Information . . . 17

3.2 Annotations . . . 18

3.2.1 Overview . . . 18

3.2.2 Supported Annotation Types . . . 19
3.2.3 Access Annotations . . . 19
3.2.4 Create Annotations . . . 20
3.2.5 Edit Annotations . . . 26
3.2.6 Update Annotation Appearances . . . 26
3.2.7 Delete Annotations . . . 27
3.2.8 Import and Export . . . 27
3.2.9 Flatten Annotations . . . 28
3.3 Forms . . . 28
3.3.1 Overview . . . 28
3.3.2 Supported Form Field Types . . . 29
3.3.3 Create Form Fields . . . 30
3.3.4 Fill Form Fields . . . 34
3.3.5 Edit Form Fields . . . 35
3.3.6 Delete Form Fields . . . 35
3.3.7 Flatten Forms . . . 35
3.3.8 Form Import and Export . . . 36
3.3.9 QR Code Form Fields . . . 37
3.4 Document Editor . . . 39
3.4.1 Overview . . . 39
3.4.2 Insert Pages . . . 40
3.4.3 Split Pages . . . 40
3.4.4 Merge Pages . . . 41
3.4.5 Delete Pages . . . 41
3.4.6 Rotate Pages . . . 41
3.4.7 Replace Pages . . . 41
3.4.8 Extract Pages . . . 41

3.4.9 Scanned Document Detection . . . 42
3.5 Security . . . 43
3.5.1 Overview . . . 43
3.5.2 PDF Permissions . . . 43
3.5.3 Background . . . 45
3.5.4 Header and Footer . . . 47
3.5.5 Bates Numbers . . . 48
3.6 Redaction . . . 49
3.6.1 Overview . . . 49
3.6.2 Redact PDFs . . . 49
3.7 Watermark . . . 50
3.7.1 Overview . . . 50
3.7.2 Add Text Watermark . . . 51
3.7.3 Add Image Watermark . . . 51
3.7.4 Delete Watermark . . . 52
3.8 Conversion . . . 52
3.8.1 PDF Standard Format Conversion . . . 52
3.8.2 PDF Standard Format Details . . . 54
3.8.3 PDF Standard Introduction . . . 55
3.9 Documentation comparison . . . 58
3.9.1 Introduction . . . 58
3.9.2 Overlay Comparison . . . 59
3.9.3 Content Comparison . . . 59
3.10 Digital Signatures . . . 60
3.10.1 Overview . . . 60
3.10.2 Concepts of Digital Signatures . . . 61
3.10.3 Create Digital Certificates . . . 63

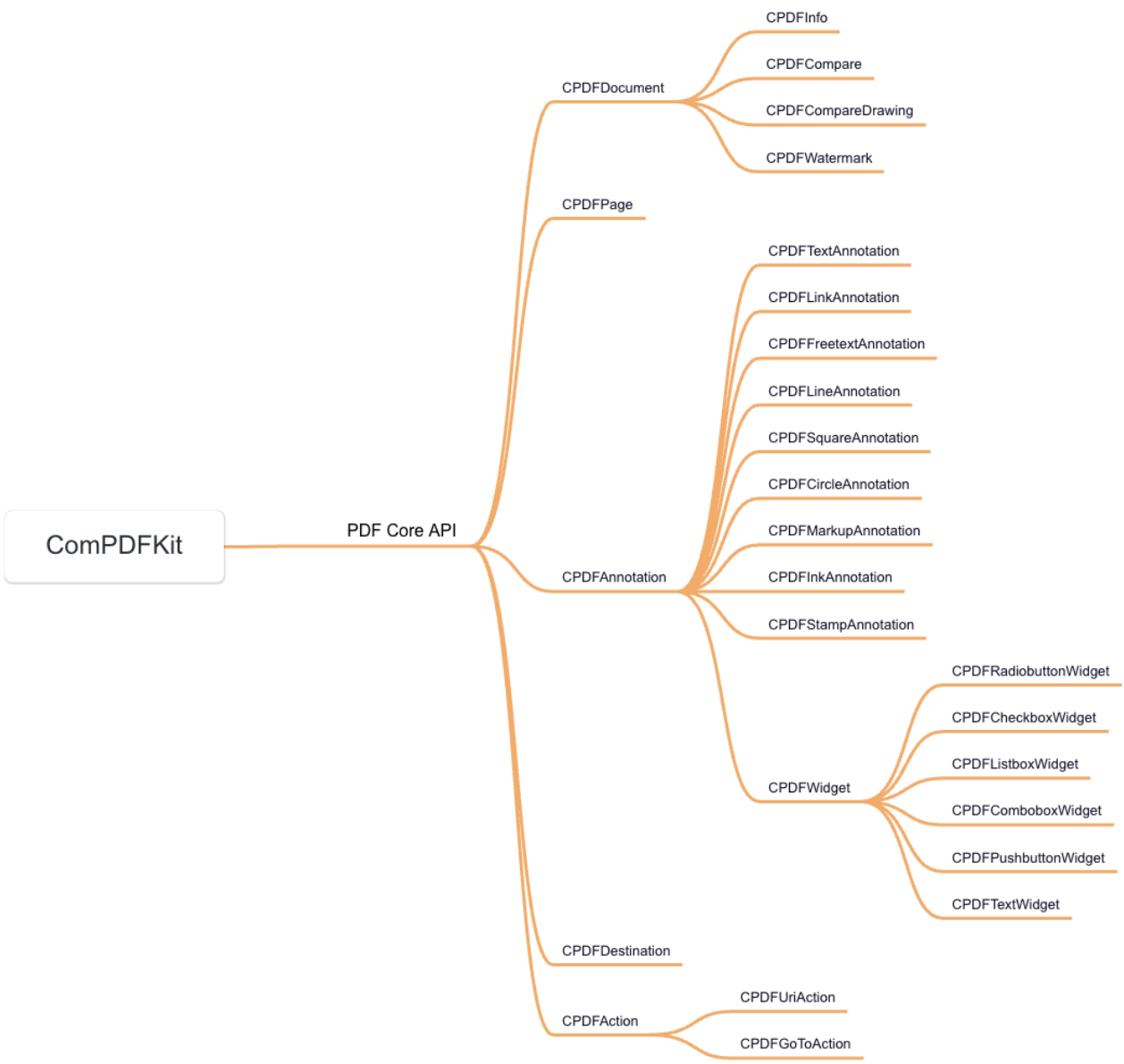
3.10.4 Create Digital Signatures . . . 63
3.10.5 Read Digital Signature Information . . . 65
3.10.6 Verify Digital Certificates . . . 66
3.10.7 Verify Digital Signatures . . . 66
3.10.8 Trust Certificate . . . 67
3.10.9 Remove Digital Signatures . . . 67
3.10.10 Invisible Digital Signatures . . . 68
3.10.11 Long-Term Validation (LTV) and PAdES . . . 72
3.10.12 AWS KMS Signing . . . 74
3.10.13 Two-Phase External Signing . . . 75
3.10.14 Signature and Document Timestamps . . . 77
3.11 Text Search . . . 79
3.11.1 Overview . . . 79
3.11.2 Search Text . . . 79
3.12 Document Compression . . . 79
3.12.1 Overview . . . 79
3.12.2 Optimization Options Configuration . . . 80
3.12.3 Image Compression Settings . . . 82
3.12.4 Execute Document Compression . . . 83
4 Support 86
4.1 Reporting Problems . . . 86
4.2 Contact Information . . . 86

1 Overview

ComPDFKit PDF SDK is a powerful PDF library for developers to add robust PDF functionality to their applications, which ships with simple-to-use java APIs to offer document viewing, creation, searching, annotation, and editing. With this SDK, even developers with limited knowledge of PDF can quickly build a professional PDF viewer with just a few lines of code on the Java platform.

1.1 ComPDFKit PDF SDK

ComPDFKit PDF SDK consists of two elements as shown in the following picture.



The element for ComPDFKit PDF SDK for Java:

- PDF Core API

  A parser for PDF documents (also known as PDF Data layer) doesn't include any UI components. If you just need to render a PDF document without displaying any content inside the PDF, you can just integrate this into your Java application.

1.2 Key Features

Annotations component offers:

- Create, edit and remove annotations, including Note, Link, Free Text, Line, Square, Circle, Highlight, Underline, Squiggly, Strikeout, Ink, and Stamp, Ink, and Sound.
- Support for annotation appearance adjustments.
- Import and export annotations to/from XFDF.
- Support for annotation flattening.

Forms component offers:

- Create, edit and remove form fields, including Push Button, Check Box, Radio Button, Text Field, Combo Box, List Box, Signature, and QR Code.
- Fill PDF Forms.
- Generate QR codes as pure vector appearances, with configurable error correction level and quiet zone.
- Support for PDF form flattening.
- Import and export form data to/from XFDF.

Document Editor component offers:

- PDF manipulation, including Split pages, Merge pages, and Merge files.
- Page edit, including Add pages, Delete pages, Insert pages, Move pages, Rotate pages, Replace pages, Exchange pages, Reverse pages, Scale pages, Crop pages, and Copy & Paste pages.
- Access document information.
- Extract images.
- Scanned document detection, detecting whether a page is a pure image page.

Security component offers:

- Encrypt and decrypt PDFs, including Permission setting and Password protected.
- Create, edit, and remove header & footer, including dates, page numbers, document name, author name, and chapter name.
- Create, edit, and remove bates numbers.
- Create, edit, and remove background that can be a solid color or an image.

Redaction component offers:

- Redact content including images, text, and vector graphics to remove sensitive information or private data, which cannot be restored once applied.
- Create redaction by selecting an area or searching for a specific text.
- Edit and save redaction annotations.

Watermark component offers:

- Create, edit, add, remove, update, and get the watermarks.
- Support text and image watermarks.

Conversion component offers:

- PDF to PDF/A, supporting PDF/A-1a, PDF/A-1b, PDF/A-2a, PDF/A-2b, PDF/A-2u, PDF/A-3a, PDF/A-3b, PDF/A-3u standards.
- PDF to PDF/X, PDF/E, PDF/UA, PDF/VT and other international standard formats.

Document Comparison component offers:

- Compare different versions of a document, including overlay comparison and content comparison.

Digital Signatures component offers:

- Sign PDF documents with digital signatures.
- Create and verify digital certificates.
- Create and verify digital signatures.
- Create self-sign digital ID and edit signature appearance.
- Support PKCS12 certificates.
- Trust certificates.
- Create invisible digital signatures with a zero-size signature widget.
- Create PAdES B/T/LT/LTA signatures and prolong LTV evidence.
- Sign through AWS KMS using the supported RSA configuration.
- Prepare and fill external signatures in two phases, including cross-process recovery.
- Request RFC 3161 tokens and append document timestamps.

Text Search component offers:

- Search for specified text content in PDF documents.
- Support case-sensitive and case-insensitive search.
- Support whole word matching search.

Document Compression component offers:

- Compress PDF documents to reduce file size.
- Support image quality control and compression optimization.
- Support removing unused resources and objects.

2 Get Started

ComPDFKit is a powerful PDF SDK. It is easy to embed ComPDFKit PDF SDK in your Java application with a few lines of Java code. Take just a few minutes and get started.

The following sections introduce the requirements, structure of the installation package, and how to make a PDF Reader/Editor in Java with ComPDFKit PDF SDK.

2.1 Requirements

Before starting, please make sure that you have already met the following prerequisites.

2.1.1 Get ComPDFKit License Key

ComPDF provides two types of license key: 30-day free trial license, and formal license.

How to Get Free Trial License

Contact our sales team and we'll send you a 30-day free trial license for ComPDFKit PDF SDK.

How to Get Formal License

ComPDFKit PDF SDK is a commercial SDK that requires a license for application release. Any documents, sample code, or source code distribution from the released package of ComPDFKit PDF SDK to any third party is prohibited.

To get formal licenses for advanced features, custom requirements, or quote inquiries, feel free to contact our sales team.

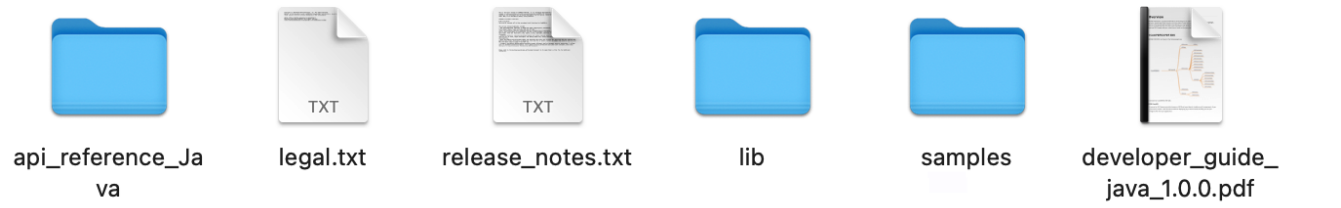
2.1.2 System Requirements

Programming Environment	Java JDK 1.8 and higher
Supported Platforms	Mac (intel & M1) or Linux (x86)

2.2 Java Package Structure

The package of ComPDFKit PDF SDK for Java includes the following files:

- "samples“ - A folder containing Java sample projects.
  - "Annotations" - A PDF viewer with full types of annotation editing, including adding annotations, modifying annotations, annotation lists, etc.
  - "Forms" - A PDF viewer with full types of forms editing, including radio button, combo box, etc.
  - "DocsEditor" - A PDF viewer with page editing, including inserting/deleting pages, extracting pages, reordering pages, etc.
  - "TestFile" - A folder containing test files.
  - "lib" - A folder containing "compdfkit-1.0-SNAPSHOT". compdfkit-1.0-SNAPSHOT is PDF Core API.
- "api_reference_Java" - API reference.
- "developer_guide_Java.pdf" - Developer guide.
- "release_notes.txt" - Release information.
- "legal.txt" - Legal and copyright information.



2.3 Apply the license Key

If you haven't get a license key, please check out how to obtain a license key.

ComPDFKit PDF SDK currently supports two authentication methods to verify license keys: online authentication and offline authentication.

Learn about: What is the authentication mechanism of ComPDFKit's license? What are the differences between Online Authentication and Offline Authentication?

2.3.1 Copying the License Key

Accurately obtaining the license key is crucial for the application of the license.

1. In the email you received, locate the XML file containing the license key.
2. Open the XML file.

Offline License:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
<platform>linux</platform>
<starttime>xxxxxxxx</starttime>
<endtime>xxxxxxxx</endtime>
<key>LICENSE_KEY</key>
</license>
```

3. Copy the LICENSE_KEY from the <key>LICENSE_KEY</key> field.

2.3.2 Apply the License Key

You can contact the ComPDFKit team to obtain a trial license. Before using any ComPDFKit PDF SDK classes, you must perform the following steps to apply the license to your application:

1. Open your project.
2. Initialize the license based on your requirements, either through online or offline authentication obtained in the previous step.
3. Initialize the license as follows:

• Offline license:

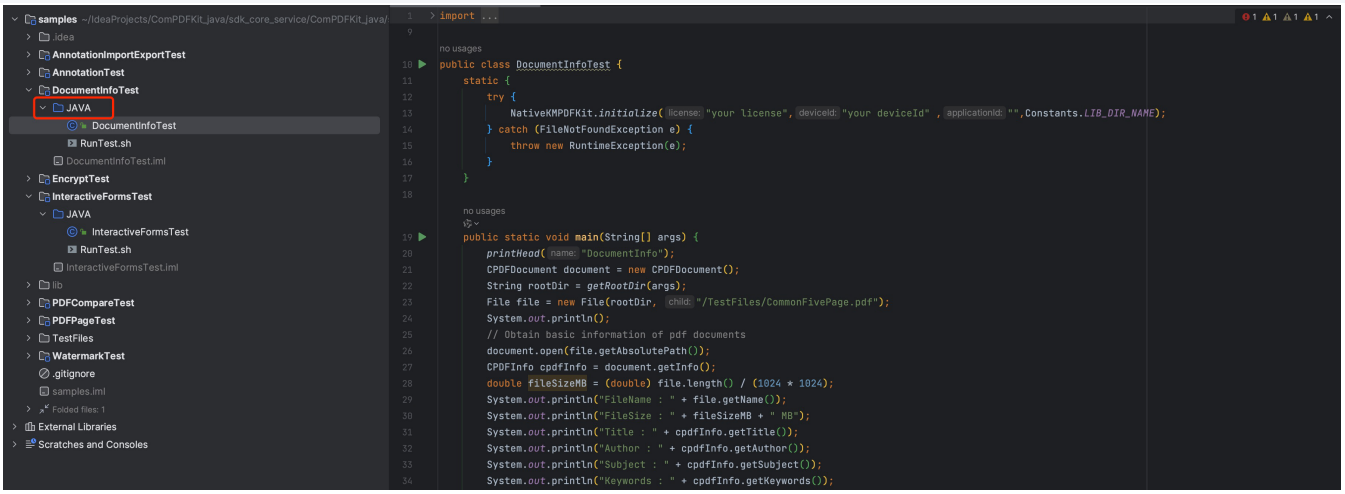
```
NativeKMPDFKit.initialize("LICENSE_KEY", "your deviceId", "", Constants.LIB_DIR_NAME);
```

2.4 How to Run a Demo

ComPDFKit PDF SDK for Java provides multiple demos for developers to learn how to call the SDK on Java applications. You can find them in the "samples" folder. In this guide, we take "DocumentInfoTest" as an example to show how to run it on Java applications.

1. Import the "samples" project on IDE.
2. Navigate to the "JAVA" folder which is in "DocumentInfoTest" file and run the sample with. For example /samples/DocumentInfoTest/JAVA.

```
./RunTest.sh
```



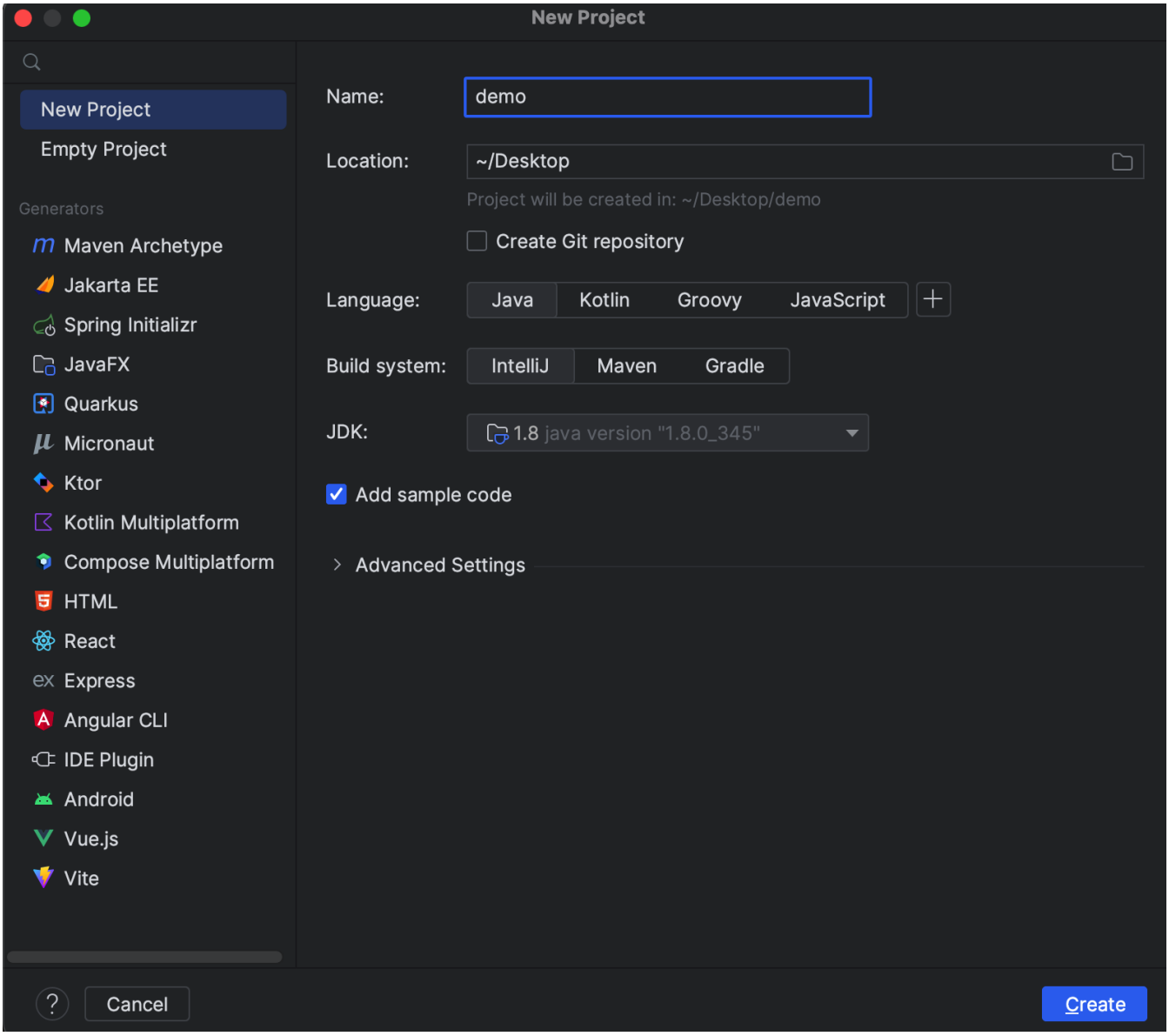
Note: This is a demo project, presenting completed ComPDFKit PDF SDK functions. The functions might be different based on the license you have purchased. Please check that the functions you choose work fine in this demo project. The output files will be in /samples/out.

2.5 How to Use ComPDFKit PDF SDK for Java in Java Applications

This section will help you to quickly get started with ComPDFKit PDF SDK to make a Java application with step-by-step instructions.

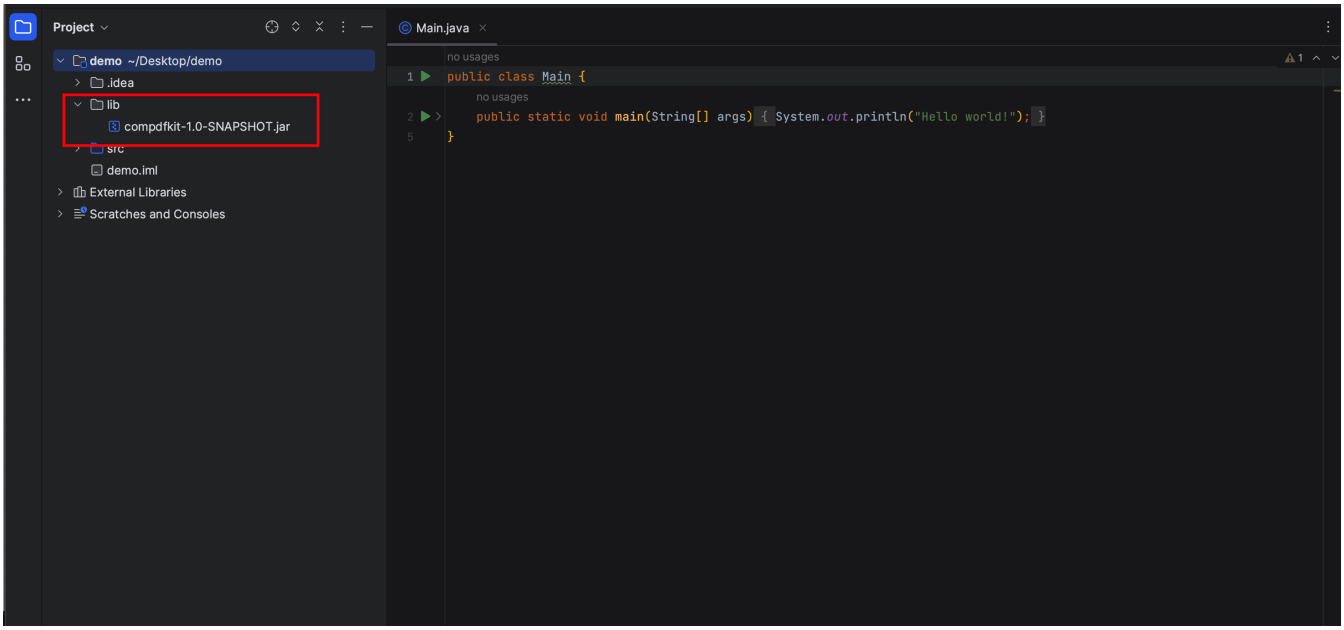
2.5.1 Create a New Project

1. Create a Java application using the IDE. Here we create a new Java project.

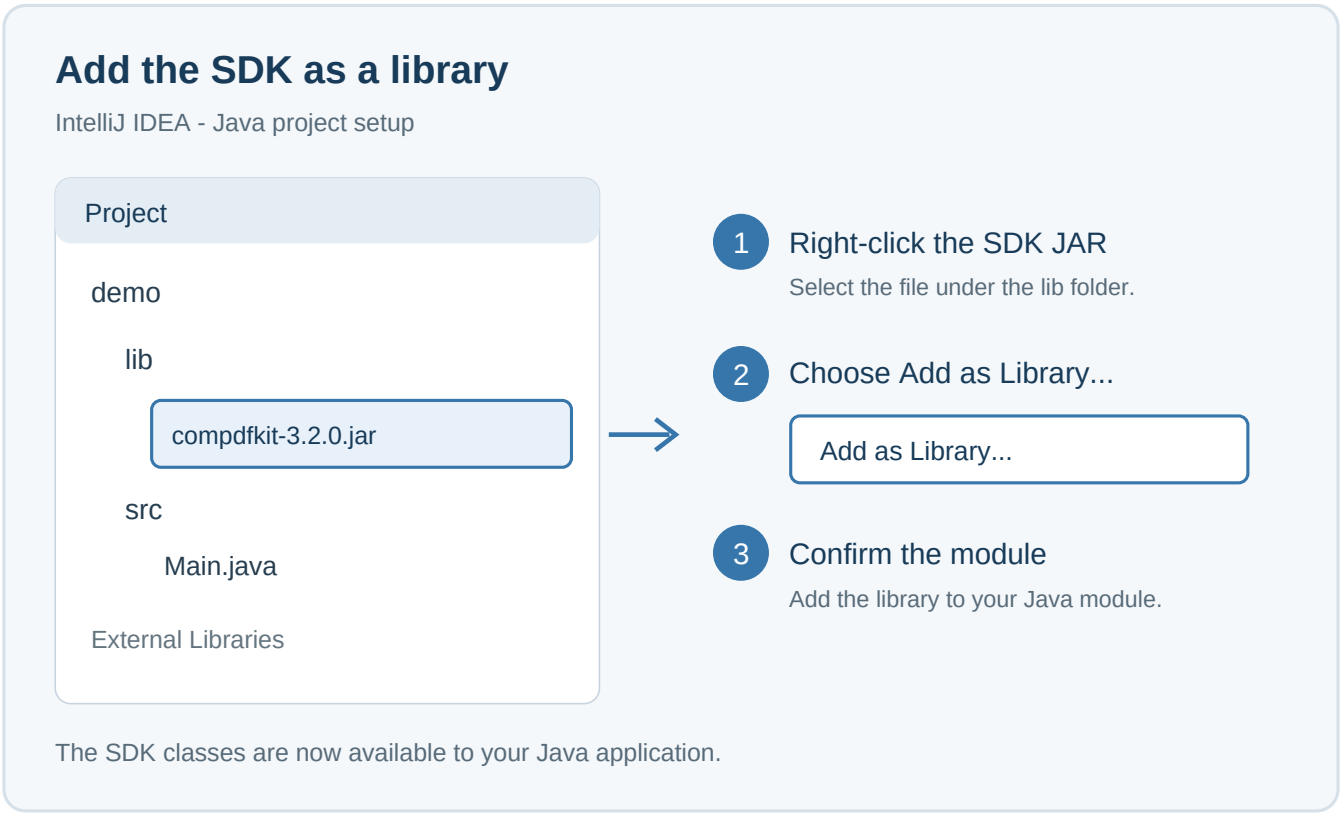


2.5.2 Add the Package of ComPDFKit PDF SDK for Java

1. The first thing we need to do is to import ComPDFKit PDF SDK for Java. Copy "lib" to the project.



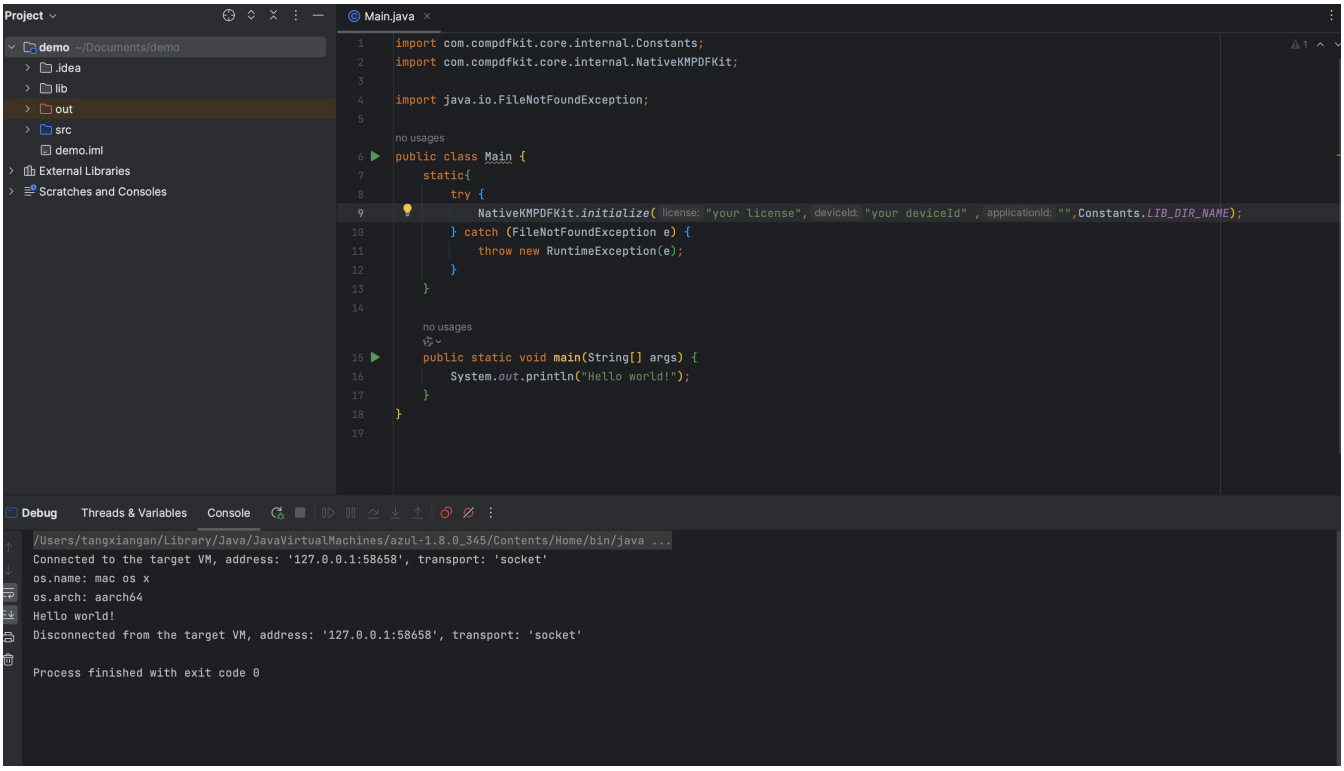
2. Right click on the SDK and select Add as Library....



3. Add the following code to Main.java:

```
static{
    try {
        NativeKMPDFKit.initialize("your license", "your deviceId", "", Constants.LIB_DIR_NAME);
    } catch (FileNotFoundException e) {
            throw new RuntimeException(e);
    }
}
```

4. Start the main method to print out the information of your computer's system version. If the information is printed successfully, you have integrated ComPDFKit PDF SDK for Java successfully!



2.5.3 Troubleshooting

1. Problems

   If you meet some other problems when integrating our ComPDFKit PDF SDK for Java, feel free to contact ComPDFKit team.

2.6 Samples

The Samples use preset parameters and documentation to call the API of ComPDFKit PDF SDK for Java for each function without UI interaction or parameter settings. This is achieved through modular code examples. The functions include creating, getting, and deleting various types of annotations and forms, extracting text and images, encrypting and decrypting documents, adding watermarks and bates numbers, and more.

These projects not only demonstrate the best practices for each function but also provide detailed introductions. The impact of each function on PDF documents can be observed in the output directory. With the help of the Samples, you can quickly learn how to use the functions you need and apply them to your projects.

Name	Description
Annotation	Print the annotation list information, set the annotations (including markup, note, ink, free text, circle, square, line annotations), and delete the annotations.
AnnotationImportExport	Export and import annotations with an xfdf file.
InteractiveForms	Print form list information, set up interactive forms (including text, checkbox, radio button, button, list, combo boxes and deleting forms), and fill out form information.
PDFPage	Manipulate PDF pages, including inserting, splitting, merging, rotating, and replacing, etc.
DocumentInfo	Display the information of PDF files like the author, created time, etc.
Encrypt	Set passwords to encrypt PDFs and set document permissions. Allow decrypting PDFs.
Watermark	Create text/image watermarks and delete watermarks.

Name	Description
Compare	Compare the differences between two documents, providing both content comparison and overlay comparison.
Flatten	Flatten PDF annotations and forms, and merge all layouts as one layout.
PDFRedact	Create redaction to remove sensitive information or private data, which cannot be viewed and searched once applied.
Background	Set color background, image background, set background properties, remove background.
ConvertPDFA	Convert PDF file to PDF/A, PDF/X, PDF/E, PDF/UA and PDF/VT format.
HeaderFooter	Add header and footer, set header and footer properties, remove header and footer.
Bates	Add Bates numbers, get Bates numbers, set Bates number properties, remove Bates numbers.
ImageDoc	Determine if a document is a scanned document, determine if a page is a scanned page.
Optimize	Document compression and optimization, configure optimization options, set image compression options.
Outline	Create outlines, read outline information, remove outlines.
TextSearch	Text search, configure search options, get search results.
QRCode	Create a QR code form field, set its content, error correction level and quiet zone, generate its vector appearance, and read the values back from the saved document.

3 Guides

If you're interested in the features mentioned in Overview section, please go through our guides to quickly add PDF viewing, annotating, and editing to your application. The following sections list some examples to show you how to add document functionalities to Java apps using the APIs of ComPDFKit PDF SDK for Java.

3.1 Basic Operations

3.1.1 Overview

There are some common basic operations when handling documents.

Guides for Basic Operations

- [Open a Document](#3.1.2 Open a Document)

  Open a local PDF document or create a new PDF document from scratch.

- [Save a Document](#3.1.3 Save a Document)

  Save the document to the original path or save it as a new document.

- [Document Information](#3.1.4 Document Information)

  View information such as the file creator, creation time, modification time, and more.

3.1.2 Open a Document

ComPDFKit supports opening local PDF documents or creating new ones.

Open a Local PDF Document

The steps to open a local PDF document using a file path are as follows:

1. Obtain the local file path.
2. Initialize a CDFDocument object using the file path.

This example shows how to open a local PDF document:

```
CPDFDocument document = new CPDFDocument();
// Open the document
CPDFDocument.PDFDocumentError error = document.open(pdfUri);
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorPassword) {
// For encrypted documents, it is necessary to use a password to decrypt them.
  error = document.open(pdfUri, "password");
}
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorSuccess) {
// The document has been successfully opened, and data can be parsed and manipulated.
} else {
 // Unable to open the PDF file. You can refer to the API documentation for specific error messages.
}
// Check whether the document has been repaired during the opening process.
if (document.hasRepaired()) {
 // The repaired document needs to be saved first before reopening. Otherwise, any edits or modifications made to the repaired
    document cannot be saved.
  // Perform the save operation.
  ...
}
```

Create a New PDF Document

This example shows how to create a new PDF document:

```
CPDFDocument document = CPDFDocument.createDocument(context);
```

By default, a newly created document doesn't contain any pages. Please refer to the "Document Editing" functionality to learn how to create new pages and add existing pages to the document.

Explanation of Open Document Status

This is the explanation of opening document status:

Error	Description
Error Code	Description
PDFDocumentErrorSuccess	Document opened successfully.
PDFDocumentErrorUnknown	Unknown error.
PDFDocumentErrorFile	File not found or cannot be opened.
PDFDocumentErrorFormat	Format Error: not a PDF or corrupted.
PDFDocumentErrorPassword	Password required or incorrect password.
PDFDocumentErrorSecurity	Unsupported security scheme.
PDFDocumentErrorPage	Page not found or content error.
PDFDocumentNotVerifyLicense	License does not permit this permission.

3.1.3 Save a Document

ComPDFKit supports incremental saving and full saving.

When the document is saved to the original path, the PDF document will be saved incrementally, meaning all changes will be appended to the file. This can significantly speed up the saving process for large files. However, it results in an increase in document size with each save.

When the document is saved to a new path, the PDF document will undergo non-incremental saving. This entails overwriting the entire document instead of appending changes at the end.

This example shows how to save a document by incremental saving and full saving :

```
// First, check if the document has been modified. If there are any changes, please save them.
if (document.hasChanges()) {
    boolean res;
    // Incrementally save the document object to the current path.
    res = document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
// Save the document object to the specified path in a non-incremental manner; `isRemoveSecurity` indicates whether to remove the
    document's password.
    //res = document.save(uri, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveRemoveSecurity);
}
// Close the document and release resources.
document.close();
```

3.1.4 Document Information

This example shows how to get document information:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFInfo info = document.getInfo();
String title = info.getTitle();          // Document title.
String author = info.getAuthor();        // Document author.
String subject = info.getSubject();      // Document subject.
String creator = info.getCreator();      // Application name that created the document
String producer = info.getProducer();    // Application name that generated PDF data
String keywords = info.getKeywords();    // Document keywords.
String creationDate = info.getCreationDate();   // Document creation date
String modificationDate = info.getModificationDate();    // Document last modified date
```

3.2 Annotations

3.2.1 Overview

Annotations allow users to highlight paragraphs, add comments, markup, sign, or stamp PDF documents without modifying the original author's content. The annotated content, along with the original text, can then be shared together.

Benefits of ComPDFKit PDF Annotation

- Comprehensive Type Support: Enables highlighting, text, freehand drawing, shapes, stamps, and more.
- Create, Edit, Delete: Perform creation, editing, and deletion operations either programmatically or directly through the UI.
- Flattened Annotations: Embed annotations permanently onto the document as images, ensuring document appearance stability and preventing further modifications.
- Annotation Events: Trigger specified workflows to achieve automation.
- Annotation Import and Export: Export annotations as XFDF templates and apply them to multiple documents.

Guides for Annotations

- [Access Annotations](#3.2.3 Access-annotations)

  Get a list of annotations and individual annotation objects within a document.

- [Create Annotations](#3.2.4 Create Annotations)

  Generate annotations of various types.

- [Edit Annotations](#3.2.5 Edit Annotations)

  Modify annotation positions, appearances, and properties.

- [Update Annotation Appearances](#3.2.6 Update Annotation Appearances)

  After edit the annotation properties, update the annotation appearance to apply the changes.

- [Delete Annotations](#3.2.7 Delete Annotations)

  Remove an annotation from the document.

- [Import and Export](#3.2.8 Import and Export)

  Export annotations as XFDF templates and apply them to multiple documents.

- [Flatten Annotations](#3.2.9 Flatten Annotations)

  Permanently embed existing annotations within the document as images.

3.2.2 Supported Annotation Types

ComPDFKit supports annotation types that adhere to PDF standards, as defined in the PDF reference. These annotations can be read and written by compliant PDF processors, including Adobe Acrobat. The supported annotation types are as follows:

Type	Description	Class Name
Text Annotation	Text annotations appear as small icons or labels. Users can expand them to view related content, making them suitable for personal notes, reminders, or comments.	CPDFTextAnnotation
Link Annotation	Link annotations enable users to directly navigate to other locations within the document or external resources, providing a richer navigation experience.	CPDFLinkAnnotation
Free Text Annotation	Free text annotations allow users to insert free-form text into PDF documents, useful for adding annotations, comments, or explanations of document content.	CPDFFreetextAnnotation
Graphics: Rectangle, Circle, Line, Arrow	This category includes shapes like rectangles, circles, lines, and arrows, used to draw graphics in the document to highlight or mark specific areas.	CPDFSquareAnnotation, CPDFCircleAnnotation, CPDFLineAnnotation
Markup: Highlight, Underline, Strikethrough, Squiggly	Add markup annotations in the PDF document to emphasize, underline, strikethrough, or add squiggly lines to specific content, such as important paragraphs, lines, words, keywords, or tables.	CPDFHighlightAnnotation, CPDFUnderlineAnnotation, CPDFStrikeoutAnnotation, CPDFSquigglyAnnotation
Stamp	Stamp annotations allow the insertion of stamps or seals into PDF documents, containing text, images, or custom designs, resembling the process of stamping on physical documents.	CPDFStampAnnotation
Ink	Draw custom shapes, icons, or doodles using handwritten or mouse-drawn strokes, enabling users to freely create or add personalized graphic elements to the document.	CPDFInkAnnotation

3.2.3 Access Annotations

The steps to access a list of annotations and annotation objects are as follows:

1. Obtain the page object.
2. Access the list of annotations from the page object.
3. Iterate through each annotation object in the list.

This example shows how to access annotations:

```
CPDFDocument document = new CPDFDocument();
document.open("fileUrl");
int pageCount = document.getPageCount();
if (pageCount > 0){
  for (int pageIndex = 0; pageIndex < pageCount; pageIndex++) {
    List<CPDFAnnotation> annotations = document.pageAtIndex(pageIndex).getAnnotations();
    if (annotations != null && !annotations.isEmpty() ){
      for (CPDFAnnotation annotation:annotations){
        if (annotation == null || !annotation.isValid()){
          continue;
        }
       // Begin operations on the acquired annotations.
      }
    }
  }
}
```

3.2.4 Create Annotations

ComPDFKit supports a wide range of annotation types, including notes, links, shapes, highlights, stamps, freehand drawings, and audio annotations, catering to diverse annotation requirements.

Create Note Annotations

Note annotations appear as small icons or labels. When clicked by the user, they can expand to display relevant annotation content. This annotation type is used for adding personal notes, reminders, or comments, allowing users to add personalized additional information to the document without affecting the readability of the original text.

The steps to create a note are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create a note annotation object on that page.
3. Set the annotation properties.
4. Update the annotation appearance to display it on the document.

This example shows how to create note annotations:

```
// Insert page numbers.
int pageNumber = 0;
// Retrieve an instance of the PDF page.
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFTextAnnotation textAnnotation = (CPDFTextAnnotation) page.addAnnot(CPDFAnnotation.Type.TEXT);
// Retrieve the actual size of the page to be inserted.
RectF pageSize = page.getSize();
RectF insertRect = new RectF(0, 0, 50, 50);
textAnnotation.setColor(Color.RED);
textAnnotation.setAlpha(255);
textAnnotation.setRect(insertRect);
textAnnotation.setContent("ComPDFKit");
textAnnotation.updateAp();
```

Create Link Annotations

Link annotations allow users to navigate directly to other locations within the document or external resources, enhancing the navigation experience.

The steps to create link annotations are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create a link annotation object on that page.
3. Use CPDFDestination to specify the destination page, for example, jumping to page 2.
4. Set the annotation properties and attach the CPDFDestination object to the annotation.

This example shows how to create link annotations:

```
// Insert page numbers.
int pageNumber = 1;
// Retrieve an instance of the PDF page.
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFLinkAnnotation linkAnnotation = (CPDFLinkAnnotation) page.addAnnot(CPDFAnnotation.Type.LINK);
RectF insertRect = new RectF(0,0,100,100);
linkAnnotation.setRect(insertRect);

// Navigate to a specified page number.
CPDFGoToAction goToAction = new CPDFGoToAction();
CPDFDestination destination = new CPDFDestination(pageNumber,0,pageSize.height(),1f);
goToAction.setDestination(document, destination);
linkAnnotation.setLinkAction(goToAction);

// Open a specific webpage.
CPDFUriAction uriAction = new CPDFUriAction();
uriAction.setUri("website");
linkAnnotation.setLinkAction(uriAction);
// Update the annotations onto the document.
linkAnnotation.updateAp();
```

Create Free Text Annotations

Free text annotations enable users to insert free-form text into PDF documents, serving the purpose of adding annotations, comments, or explanations to document content.

The steps to create a free text annotation are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create a free text annotation object on that page.
3. Set the annotation properties.
4. Update the annotation appearance to display it on the document.

This example shows how to create free text annotations:

```
// Insert page numbers.
int pageNumber = 0;
// Retrieve an instance of the PDF page.
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFFreetextAnnotation freetextAnnotation = (CPDFFreetextAnnotation) page.addAnnot(CPDFAnnotation.Type.FREETEXT,
    document);
RectF insertRect = new RectF(0,0,100,100);
freetextAnnotation.setRect(insertRect);
// Set annotation properties
freetextAnnotation.setContent("Hello World!");
freetextAnnotation.setFreetextAlignment(CPDFFreetextAnnotation.Alignment.ALIGNMENT_LEFT);
CPDFTextAttribute textAttribute = freetextAnnotation.getFreetextDa();
textAttribute.setColor(Color.YELLOW.getRGB());
textAttribute.setFontSize(14);
textAttribute.setFontName("fontName");
freetextAnnotation.setFreetextDa(textAttribute);
freetextAnnotation.updateAp();
// Update the annotations onto the document.
freetextAnnotation.updateAp();
```

Create Shape Annotations

Graphic annotations encompass shapes such as rectangles, circles, lines, and arrows, used to draw attention to or highlight specific areas in a document, conveying information that may be challenging to describe with text alone.

The steps to create graphic annotations are as follows:

1. Obtain the page object where the annotations need to be created.

2. Sequentially create rectangle, circle, and line annotations on that page.
3. Set the annotation properties.
4. Sequentially update the annotation appearance to display them on the document.

This example shows how to create shape annotations:

```
// Insert page numbers.
int pageNumber = 0;
// Retrieve an instance of the PDF page.
CPDFPage page = document.pageAtIndex(pageNumber);
// Create a rectangle annotation object on the page.
CPDFSquareAnnotation squareAnnotation = (CPDFSquareAnnotation) page.addAnnot(CPDFAnnotation.Type.SQUARE, document);
RectF insertRect = new RectF(0,0,100,100);
squareAnnotation.setRect(insertRect);
// Set annotation properties.
squareAnnotation.setBorderColor(Color.YELLOW.getRGB());
CPDFBorderStyle borderStyle = new CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid, 10, new float[]{8.0F, 0F});
borderStyle.setBorderWidth(10F);
squareAnnotation.setBorderStyle(borderStyle);
squareAnnotation.setBorderAlpha(255);
squareAnnotation.setFillColor(Color.RED.getRGB());
squareAnnotation.setFillAlpha(255);
// Update the annotations onto the document.
squareAnnotation.updateAp();

// Create a circular annotation object on the page.
CPDFCircleAnnotation circleAnnotation = (CPDFCircleAnnotation) page.addAnnot(CPDFAnnotation.Type.CIRCLE, document);
RectF insertRect = new RectF(0,0,100,100);
circleAnnotation.setRect(insertRect);
// Set annotation properties.
circleAnnotation.setBorderColor(Color.YELLOW.getRGB());
CPDFBorderStyle borderStyle = new CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid, 10, new float[]{8.0F, 0F});
borderStyle.setBorderWidth(10F);
circleAnnotation.setBorderStyle(borderStyle);
circleAnnotation.setBorderAlpha(255);
circleAnnotation.setFillColor(Color.RED.getRGB());
circleAnnotation.setFillAlpha(255);
// Update the annotations onto the document.
circleAnnotation.updateAp();

// Create a circular annotation object on the page.
CPDFLineAnnotation lineAnnotation = (CPDFLineAnnotation) page.addAnnot(CPDFAnnotation.Type.LINE, document);
float lineWidth = 10f;
PointF startPoint = new PointF(0,0);
PointF endPoint = new PointF(200,200);
RectF area = new RectF(0,0,200,200);
lineAnnotation.setRect(area);
lineAnnotation.setLinePoints(startPoint, endPoint);
// Set annotation properties.
lineAnnotation.setLineType(CPDFLineAnnotation.LineType.LINETYPE_NONE, CPDFLineAnnotation.LineType.LINETYPE_ARROW);
CPDFBorderStyle borderStyle = new CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid,lineWidth,null);
lineAnnotation.setBorderStyle(borderStyle);
lineAnnotation.setBorderAlpha(255);
lineAnnotation.setBorderColor(Color.RED.getRGB())

// Update the annotations onto the document.
lineAnnptation.updateAp();
```

Explanation of Line Types

Name	Description
LINETYPE_UNKNOWN	Non-standard or invalid line segment endpoint.
LINETYPE_NONE	No line segment endpoint.
LINETYPE_ARROW	Two short lines intersect at a sharp angle, forming an open arrow.

Name	Description
LINETYPE_CLOSEDARROW	Two short lines intersect at a sharp angle, similar to style one, and are connected by a third line, forming a triangular closed arrow filled with the interior color of the annotation.
LINETYPE_SQUARE	A square filled with the interior color of the annotation.
LINETYPE_CIRCLE	A circle filled with the interior color of the annotation.
LINETYPE_DIAMOND	A diamond shape filled with the interior color of the annotation.
LINETYPE_BUTT	A short line perpendicular to the line itself, located at the endpoint.
LINETYPE_ROPENARROW	Two short lines in opposite directions.
LINETYPE_RCLOSEDARROW	A triangular closed arrow in opposite directions.
LINETYPE_SLASH	A short line, located at the endpoint, rotated approximately 30 degrees clockwise from the direction perpendicular to the line itself.

Create Markup Annotations

Incorporate annotations in a PDF document to highlight, emphasize, or explain specific content, such as important paragraphs, lines, words, keywords, tables, etc. ComPDFKit provides four types of markup annotations: highlight, underline, squiggly line, and strikethrough.

The steps to create a markup annotation are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create a text object through the page object.
3. Use the text object to identify the location of the text to be marked.
4. Create the corresponding markup object on that page.
5. Set the properties of the markup object.
6. Update the annotation appearance to display it on the document.

This example shows how to create markup annotations:

```
// First, retrieve a TextSelection array from a specific region on the page to select text.
CPDFPage pdfPage = douc.pageAtIndex(1);
CPDFTextPage pdfTextPage = pdfPage.getTextPage();
RectF selectRect = new RectF(0f, 0f, 500f, 500f);
// Then, add a highlight annotation to the specific region for highlighting.
highlightAnnotation.setQuadRects(selectRect);
highlightAnnotation.setMarkedText("text");
highlightAnnotation.setRect(annotRect);
highlightAnnotation.updateAp();
```

Create Stamp Annotations

Stamp annotations are used to identify and validate the source and authenticity of a document. ComPDFKit supports standard stamps, text stamps, and image stamps.

The steps to create a stamp annotation are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create the corresponding stamp on that page.
3. Set the properties of the stamp.
4. Update the annotation appearance to display it on the document.

This example shows how to create stamp annotations:

```
// Retrieve the page object for creating annotations.
int pageNumber = 0;
CPDFPage pdfPage = document.pageAtIndex(pageNumber);

// Create a standard stamp.
int yOffset = 50;
float lastOffset = 0;
for (int i = 0; i < CPDFStampAnnotation.StandardStamp.values().length; i++) {
    CPDFPage page = document.pageAtIndex(4);
    CPDFStampAnnotation.StandardStamp standardStamp = CPDFStampAnnotation.StandardStamp.values()[i];
    if (standardStamp == null || standardStamp == CPDFStampAnnotation.StandardStamp.UNKNOWN) {
        continue;
    }
    // Add Standard stamp
    CPDFStampAnnotation standard = (CPDFStampAnnotation) page.addAnnot(CPDFAnnotation.Type.STAMP, document);
    if (standard == null) {
        continue;
    }
    standard.setStandardStamp(standardStamp);
    RectF pageSize = page.getSize();
    RectF insertRect = standard.getRect();
    float defaultWidth = 100F;
    int x = 50;
    if (i == 10) {
        lastOffset = 50;
    }
    if (i >= 10) {
        x = 150;
    }
    yOffset = (int) lastOffset + 10;
    PointF vertex = new PointF(x, yOffset);
    insertRect.set(vertex.x, vertex.y, vertex.x + defaultWidth, vertex.y + defaultWidth * Math.abs(insertRect.height() / insertRect.width()));
    lastOffset = insertRect.bottom;
    standard.setRect(insertRect);
    standard.updateAp();
}

// Create a text stamp.
CPDFPage page = document.pageAtIndex(4);
CPDFStampAnnotation stampAnnotation = (CPDFStampAnnotation) page.addAnnot(CPDFAnnotation.Type.STAMP, document);
SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
String date = df.format(new Date());
stampAnnotation.setTextStamp(new CPDFStampAnnotation.TextStamp(
        "ComPDFKit", date, CPDFStampAnnotation.TextStampShape.TEXTSTAMP_RECT.id,
        CPDFStampAnnotation.TextStampColor.TEXTSTAMP_GREEN.id));
RectF insertRect = stampAnnotation.getRect();
insertRect.set( insertRect);
float defaultWidth = 150f;
PointF vertex = new PointF(300, 50);
insertRect.set(vertex.x, vertex.y, vertex.x + defaultWidth, vertex.y + defaultWidth * Math.abs(insertRect.height() / insertRect.width()));
stampAnnotation.setRect(insertRect);
stampAnnotation.updateAp();

// Create an text signature stamp.
CPDFStampAnnotation standard = (CPDFStampAnnotation) page.addAnnot(CPDFAnnotation.Type.STAMP, document);
String imagePath = rootDir + "/TestFiles/ComPDFKit.png";
RectF imageInsertRect = new RectF(457, 626.5f, 513, 570.5f);
standard.setRect(imageInsertRect);
if (imagePath.endsWith("png")) {
    File imageFile = new File(imagePath);
    BufferedImage image = null;
    try {
        image = ImageIO.read(imageFile);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    int[] pixel = getPixel(image);
    standard.updateApWithBitmap(pixel, image.getWidth(), image.getHeight());
} else {
```

```
    standard.setImageStamp(imagePath);
    standard.updateAp();
}
```

Create Ink Annotations

Ink annotations provide a direct and convenient method for drawing annotations.

The steps to create a ink annotation are as follows:

1. Obtain the page object where the annotation needs to be created.
2. Create a ink annotation on that page.
3. Set the path taken by the ink annotation.
4. Set other properties of the annotation.
5. Update the annotation appearance to display it on the document.

This example shows how to create ink annotations:

```
// Retrieve the page object for creating annotations.
int pageNumber = 0;
CPDFPage page = document.pageAtIndex(pageNumber);
// Set the path traced by the freehand annotation.
CPDFPage page1 = document.pageAtIndex(0);
ArrayList<ArrayList<PointF>> mDrawing = new ArrayList<>();
ArrayList<PointF> arc1 = new ArrayList<>();
arc1.add(new PointF(100, 100));
arc1.add(new PointF(110, 110));
arc1.add(new PointF(120, 120));
mDrawing.add(arc1);
ArrayList<PointF> arc2 = new ArrayList<>();
arc2.add(new PointF(115, 115));
arc2.add(new PointF(130, 130));
arc2.add(new PointF(160, 160));
mDrawing.add(arc2);
float scaleValue = 1F;
float borderWidth = 5F;
// Create a ink annotation on the page.
CPDFInkAnnotation inkAnnotation = (CPDFInkAnnotation) page1.addAnnot(CPDFInkAnnotation.Type.INK, document);
inkAnnotation.setColor(Color.RED.getRGB());
inkAnnotation.setAlpha(255);
inkAnnotation.setBorderWidth(borderWidth);
RectF rect = null;
RectF size = document.getPageSize(0);
if (size.isEmpty()) {
    return;
  }
int lineCount = mDrawing.size();
PointF[][] path = new PointF[lineCount][];
for (int lineIndex = 0; lineIndex < lineCount; lineIndex++) {
  ArrayList<PointF> line = mDrawing.get(lineIndex);
  int pointCount = line.size();
  PointF[] linePath = new PointF[pointCount];
  for (int pointIndex = 0; pointIndex < pointCount; pointIndex++) {
      PointF point = line.get(pointIndex);
       /****** Calculate the minimum bounding rectangle for the path ******/
      if (rect == null) {
        rect = new RectF(point.x, point.y, point.x, point.y);
         } else {
            rect.union(point.x, point.y);
      }
      /****** Calculate the coordinates transformed to the page and stored in the linePath collection ******/
       linePath[pointIndex] = point;
 }
  path[lineIndex] = linePath;
}
float dx = borderWidth / scaleValue / 2;
```

```
rect.inset(-dx, -dx);
rect.set(rect);
inkAnnotation.setInkPath(path);
inkAnnotation.setRect(rect);
inkAnnotation.updateAp();
```

Creating Reply Annotations

Reply annotations are used to reply to comments on annotations.

The steps to create a reply annotation are as follows:

1. Get the page object and comment for which you need to create a reply.
2. Create the reply comment on that page annotation.
3. set other properties of the annotation.

This example shows how to create reply annotations:

```
// Add reply annotation
CPDFPage page = document.pageAtIndex(0);
CPDFFreetextAnnotation freetextAnnotation = (CPDFFreetextAnnotation) page.getAnnotations().get(0);      CPDFAnnotation
   replyAnnotation = freetextAnnotation.createReplyAnnotation();
if (replyAnnotation.isReplyAnnot()) {
   Date date = new Date();
   CPDFDate cpdfDate = new CPDFDate(date.getYear(), date.getMonth(), date.getDate(), date.getHours(), date.getMinutes(),
   date.getSeconds(), 480);
   replyAnnotation.setCreationDate(cpdfDate);
   replyAnnotation.setRecentlyModifyDate(cpdfDate);
   replyAnnotation.setTitle("Guest");
   replyAnnotation.setContent("ComPDFKit Reply");
}
```

3.2.5 Edit Annotations

The steps to edit an annotation are as follows:

1. Obtain the page object where the annotation needs to be edited.
2. Access the list of annotations on that page.
3. Locate the desired annotation in the annotation list and convert it.
4. Set the properties to the annotation object.
5. Update the annotation appearance to display it on the document.

This example shows how to edit annotations:

```
CPDFPage page = document.pageAtIndex(0);
List<CPDFAnnotation> annotations = page.getAnnotations();
if (annotations != null && annotations.size() > 0) {
  CPDFAnnotation annotation = annotations.get(0);
  if (annotation.getType() == CPDFAnnotation.Type.FREETEXT) {
    CPDFFreetextAnnotation freetextAnnotation = (CPDFFreetextAnnotation) annotation;
    CPDFTextAttribute textAttribute = new CPDFTextAttribute("fontname", 20, Color.YELLOW.getRGB());
    freetextAnnotation.setFreetextDa(textAttribute);
    freetextAnnotation.updateAp();
  }
}
```

3.2.6 Update Annotation Appearances

Annotations may include properties describing their appearance, such as annotation color or shape. However, these properties do not guarantee consistent display across different PDF readers. To address this issue, each annotation can define an appearance stream applied for rendering.

When modifying annotation properties, you must invoke the UpdateAp() method within the CPDFAnnotation class:

```
public boolean updateAp();
```

Setting a custom appearance stream for annotations is straightforward. This operation is commonly performed in stamp annotations, particularly because they don't have other properties. Stamp annotations used in this manner are often referred to as image annotations.

3.2.7 Delete Annotations

The steps to delete an annotation are as follows:

1. Obtain the page object where the annotation needs to be deleted.
2. Access the list of annotations on that page.
3. Locate the desired annotation in the annotation list.
4. Delete the identified annotation.

This example shows how to delete annotations:

```
// Retrieve the page object for annotating.
CPDFPage page = document.pageAtIndex(0);
// Retrieve the list of annotations for the page.
List<CPDFAnnotation> annotations = page.getAnnotations();
if (annotations != null && annotations.size() > 0) {
  // Locate the desired annotation in the list and remove it.
  page.deleteAnnotation(annotations.get(0));
}
```

3.2.8 Import and Export

The methods for importing and exporting XFDF annotations allow users to save and restore annotations and form data without altering the original PDF document, facilitating the sharing and processing of documents across different editors or platforms.

Import Annotations

When importing annotations via XFDF, a temporary file directory is created. It is necessary to specify both the XFDF path and the temporary file path during the annotation import.

This example shows how to import annotations:

```
//the path of xfdf
String xfdfPath = "/xfdf/exportXFDF.xfdf";
//the transfer path of audio and video files
String tmpxfdfPath = context.getCacheDir().getAbsolutePath() + "/tmp";
document.importAnnotations(xfdfPath,tmpxfdfPath);
```

Export Annotations

When exporting annotations via XFDF, a temporary file directory is generated. It is essential to specify both the XFDF path and the temporary file path during annotation export.

This example shows how to export annotations:

```
//the path of xfdf
String xfdfPath = "/xfdf/exportXFDF.xfdf";
//the transfer path of audio and video files
String tmpxfdfPath = context.getCacheDir().getAbsolutePath() + "/tmp";
document.exportAnnotations(xfdfPath,tmpxfdfPath);
```

What is XFDF?

XFDF (XML Forms Data Format) is an XML format used to describe and transmit PDF form data. It is commonly used in conjunction with PDF files to store and pass values, states, and operations of form fields.

An XFDF file contains data corresponding to a PDF form, including the names, values, options, and formats of form fields.

XFDF serves as a format for describing form data and does not encompass the PDF file itself. It is employed for storing and transmitting form data, facilitating interaction and sharing between different systems and applications.

3.2.9 Flatten Annotations

Annotation flattening refers to converting editable annotations into non-editable, non-modifiable static images or plain text forms. When the annotations are flattened, all editable elements of the entire document (including comments and forms) will be flattened, so the annotation flattening is also known as document flattening.

This example shows how to flatten annotations:

```
// Open the document from the file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath, passwod);
// Flatten all pages of the document.
boolean result = document.flattenAllPages(CPDFPage.PDFFlattenOption.FLAT_NORMALDISPLAY);
// Save the document.
if (result) {
  document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveIncremental);
}
```

What is Document Flattening?

Document flattening refers to the process of converting editable elements, such as annotations, form fields, or layers, in a PDF document into non-editable, static images, or pure text. The purpose of this process is to lock the final state of the document, eliminating editable elements.

Document flattening is typically applied in the following contexts:

1. Content Protection: Flattening can be used to protect document content, ensuring that the document remains unaltered during distribution or sharing. This is crucial for maintaining document integrity and confidentiality.
2. Form Submission: In form processing, flattening can convert user-filled form fields and annotations into static images for easy transmission, archiving, or printing, while preventing modifications to the form content in subsequent stages.
3. Compatibility and Display: Some PDF readers or browsers may encounter issues with displaying and interacting with PDF documents that contain numerous annotations or layers. Document flattening helps address these compatibility issues, enhancing the visual representation of documents in various environments.
4. File Size Reduction: Flattened documents typically have reduced file sizes since editable elements are converted into static images or text, eliminating the need to store additional data for editing information.

3.3 Forms

3.3.1 Overview

The Form (or AcroForm) feature allows users to create interactive form fields in a PDF document, enabling other users to provide information by filling out these fields. Essentially, PDF form fields are a type of PDF annotation known as Widget annotations. They are utilized to implement interactive form elements such as buttons, checkboxes, combo boxes, and more.

As PDF is an electronic format, it provides advantages that traditional paper forms do not have. For instance, users can edit information that has already been entered. Additionally, document creators can distribute PDF forms over

the internet, restrict the content and format entered by users, as well as programmatically extract and categorize theinformation filled in by users.

Benefits of ComPDFKit Form

- Full Types Supported: Supports all form field types, properties, and appearance settings.
- Create, Edit, Delete Form Fields: Perform creation, editing, and deletion operations programmatically or directly through the UI.
- Fill Form Fields: Seamlessly fill form fields using the CPDFReaderView or automatically fill them programmatically.
- Form Events: Trigger specified workflows, enabling automation.
- Form Flattening: Permanently adds forms to the document as images, ensuring document appearance stability and preventing further modifications.

Guides for Forms

- [Create Form Fields](#3.3.3 Create Form Fields)

  Create various interactive form fields.

- [Fill Form Fields](#3.3.4 Fill Form Fields)

  Add content to form fields.

- [Edit Form Fields](#3.3.4 Fill Form Fields)

  Edit form fields.

- [Delete Form Fields](#3.3.6 Delete Form Fields)

  Delete form fields.

- [Flatten Forms](#3.3.7 Flatten Forms)

  Flatten the form to render it in a fixed and non-editable appearance.

- QR Code Form Fields

  Encode text into a QR code and draw it as a vector appearance.

3.3.2 Supported Form Field Types

ComPDFKit supports various form field types compliant with the PDF standard, which are readable and writable by various programs, including Adobe Acrobat and other PDF processors that adhere to the standard. The supported form field types are as follows:

Type	Description	Class Name
Check Box	Select one or more options from predefined choices.	CPDFCheckBoxWidget
Radio Button	Select one option from predefined choices.	CPDFRadioButtonWidget
Push Button	Create custom buttons on the PDF document that acts when pressed.	CPDFPushButtonWidget
List Box	Select one or more options from a predefined list.	CPDFListBoxWidget
Combo Box	Select one option from a drop-down list of available text options.	CPDFComboBoxWidget
Text	Input text content such as name, address, email, etc.	CPDFTextWidget
Signature	Digitally sign or electronically sign the PDF document	CPDFSignatureWidget

Type	Description	Class Name
QR Code	Encode UTF-8 text into a QR code drawn as a vector appearance.	CPDFQRCodeWidget

3.3.3 Create Form Fields

Create Text Fields

Text fields allow users to input text in a designated area, commonly used for collecting user information or filling out forms.

The steps to create a text field are as follows:

1. Obtain the page object from CPDFDocument where the text field needs to be created.
2. Create a text field on the page object.
3. Set the position and other properties of the text field.

This example shows how to create a text field:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
int pageNumber = 0;
CPDFPage page = document.pageAtIndex(pageNumber);
RectF pageSize = document.getPageSize(pageNumber);
RectF singleLineTextRect = new RectF(28, 32, 237, 75);
//Insert a single-line TextField.
CPDFTextWidget textField = (CPDFTextWidget) page.addFormWidget(CPDFWidget.WidgetType.Widget_TextField,document);
textField.setRect(singleLineTextRect);
textField.setFieldName("TextField1");
textField.setText("Basic Text Field");
textField.setFontColor(Color.BLACK.getRGB());
textField.setFontSize(15);
textField.setFillColor(Color.RED.getRGB());
textField.setBorderColor(Color.YELLOW.getRGB());
textField.setBorderWidth(5F);
textField.updateAp();
```

Create Buttons

Buttons allow users to perform actions on PDF pages, such as page navigation and hyperlink jumps.

The steps to create a button are as follows:

1. Obtain the page object from CPDFDocument.
2. Create a button on the page object.
3. Set the button's position and appearance properties.
4. Create and set the text properties of the button.
5. Create and set the actions of the button.
6. Update the appearance of the button.

This example shows how to create a button:

```
CPDFDocument document = CPDFDocument();
document.open(pdfPath);
RectF pageSize = document.getPageSize(0);
CPDFPage cpdfPage = document.pageAtIndex(0);

// Create a button for navigating to the second page.
RectF pushButton1Rect = new RectF(267F, 203F, 401F, 235F);
CPDFPushbuttonWidget pushButtonWidget1 = (CPDFPushbuttonWidget)
```

```
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_PushButton,document);
pushButtonWidget1.setRect(pushButton1Rect);
pushButtonWidget1.setFieldName("PushButton1");
pushButtonWidget1.setFontColor(Color.BLACK.getRGB());
pushButtonWidget1.setFontSize(15);
pushButtonWidget1.setButtonTitle("PushButton");
CPDFGoToAction goToAction = new CPDFGoToAction();
CPDFDestination destination = new CPDFDestination(1, 0, 842, 0F);
goToAction.setDestination(document, destination);
pushButtonWidget1.setButtonAction(goToAction);
pushButtonWidget1.updateAp();

// Create a button for navigating to a web page.
RectF pushButton2Rect = new RectF(433F, 203F, 567F, 235F);
CPDFPushbuttonWidget pushButtonWidget2 = (CPDFPushbuttonWidget)
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_PushButton,document);
pushButtonWidget2.setRect(pushButton2Rect);
pushButtonWidget2.setFieldName("PushButton2");
pushButtonWidget2.setFontColor(Color.BLACK.getRGB());
pushButtonWidget2.setFontSize(15);
pushButtonWidget2.setButtonTitle("PushButton");
CPDFUriAction uriAction = new CPDFUriAction();
uriAction.setUri("https://www.compdf.com/");
pushButtonWidget2.setButtonAction(uriAction);
pushButtonWidget2.updateAp();
```

Create List Boxes

List boxes enable users to choose one or multiple items from a predefined list, providing a convenient data selection feature.

The steps to create a list box are as follows:

1. Obtain the page object from CPDFDocument where the list box needs to be created.
2. Create a list box on the page object.
3. Set the position of the list box.
4. Add list items to the list box.
5. Set the properties of the list box.

This example shows how to create a list box:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF listBoxRect = new RectF(267F, 32F, 567F, 138F);
CPDFListboxWidget listBoxWidget = (CPDFListboxWidget) cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_ListBox,docu
    ment);
listBoxWidget.setRect(listBoxRect);
listBoxWidget.setFieldName("ListBox1");
listBoxWidget.setFillColor(Color.WHITE.getRGB());
listBoxWidget.setBorderColor(Color.BLACK.getRGB());
listBoxWidget.setBorderWidth(2);
CPDFWidgetItem[] listBoxItems = new CPDFWidgetItem[]{
  new CPDFWidgetItem("List Box No.1", "List Box No.1"),
  new CPDFWidgetItem("List Box No.2", "List Box No.2"),
  new CPDFWidgetItem("List Box No.3", "List Box No.3"),
};
listBoxWidget.setOptionItems(listBoxItems);
listBoxWidget.setSelectedIndexes(new int[]{1});
listBoxWidget.updateAp();
```

Create Signature Fields

Signature fields allow users to insert digital or electronic signatures into a document, verifying the authenticity and integrity of the document.

The steps to create a signature field are as follows:

1. Obtain the page object from CPDFDocument where the signature field needs to be added.
2. Create a signature field on the page object.
3. Set the position of the signature field.
4. Set the properties of the signature field.

This example shows how to create a signature field:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF signatureRect = new RectF(28, 206, 237, 301);
CPDFSignatureWidget signatureWidget = (CPDFSignatureWidget)
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_SignatureFields,document);
signatureWidget.setFieldName("Signature1");
signatureWidget.setFillColor(Color.WHITE.getRGB());
signatureWidget.setBorderColor(Color.BLACK.getRGB());
signatureWidget.setBorderWidth(2F);
signatureWidget.setRect(signatureRect);
signatureWidget.updateAp();
```

Create Check Boxes

Check boxes allow users to indicate the state of an option by checking or unchecking it.

The steps to create a check box are as follows:

1. Obtain the page object from CPDFDocument where the checkbox needs to be added.
2. Create a checkbox on the page object.
3. Set the position of the checkbox.
4. Set the properties of the checkbox.

This example shows how to create a checkbox:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF checkBoxRect = new RectF(267, 251, 299, 283);
CPDFCheckboxWidget checkboxWidget = (CPDFCheckboxWidget) cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_Che
    ckBox,document);
checkboxWidget.setRect(checkBox1);
checkboxWidget.setFieldName("CheckBox1");
checkboxWidget.setFillColor(Color.parseColor("#CCE5E5FF").getRGB());
checkboxWidget.setBorderColor(Color.BLACK.getRGB());
checkboxWidget.setBorderWidth(2F);
checkboxWidget.setChecked(false);
checkboxWidget.updateAp();
```

Create Radio Buttons

Radio buttons allow users to select a unique option from a predefined group of options.

The steps to create a radio button are as follows:

1. Obtain the page object from CPDFDocument where the radio button needs to be added.
2. Create a radio button on the page object.

3. Set the position of the radio button.
4. Set the properties of the radio button.

This example shows how to create a radio button:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF radioButtonRect = new RectF(385F, 251F, 424F, 290F);
CPDFRadiobuttonWidget radiobuttonWidget = (CPDFRadiobuttonWidget)
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_RadioButton,document);
radiobuttonWidget.setRect(radioButtonRect);
radiobuttonWidget.setFieldName("RadioButton");
radiobuttonWidget.setCheckStyle(CPDFWidget.CheckStyle.CK_Circle);
radiobuttonWidget.setFillColor(Color.decode("#CCE5E5FF").getRGB());
radiobuttonWidget.setBorderColor(Color.BLACK.getRGB());
radiobuttonWidget.setBorderWidth(2F);
radiobuttonWidget.setChecked(false);
radiobuttonWidget.setFieldName("RadioButton1");
radiobuttonWidget.updateAp();
```

Create Combo Boxes

A combo box is an area where a selected item from the dropdown will be displayed.

The steps to create a combo box are as follows:

1. Obtain the page object from CPDFDocument where the combo box needs to be added.
2. Create a combo box on the page object.
3. Add list items to the combo box.
4. Set the properties of the combo box.

This example shows how to create a combo box:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF comboBoxRect = new RectF(267F, 143F, 567F, 189F);
CPDFComboboxWidget comboBoxWidget = (CPDFComboboxWidget)
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_ComboBox,document);
comboBoxWidget.setRect(comboBoxRect);
comboBoxWidget.setFieldName("ComboBox1");
CPDFWidgetItem[] comboBoxItems = new CPDFWidgetItem[]{
  new CPDFWidgetItem("Combo Box No.1", "Combo Box No.1"),
  new CPDFWidgetItem("Combo Box No.2", "Combo Box No.2"),
  new CPDFWidgetItem("Combo Box No.3", "Combo Box No.3"),
};
comboBoxWidget.setOptionItems(comboBoxItems, new int[]{1});
comboBoxWidget.updateAp();
```

Create QR Code Fields

QR code fields encode a piece of UTF-8 text and draw it on the page as a black-and-white vector appearance, so the result stays sharp at any zoom level and does not depend on an embedded image.

The steps to create a QR code field are as follows:

1. Obtain the page object from CPDFDocument where the QR code needs to be added.
2. Create a QR code field on the page object.
3. Set the position of the QR code field.

4. Set the content and, if needed, the error correction level and quiet zone.
5. Generate the QR code appearance.
6. Register the field in the document AcroForm fields.

This example shows how to create a QR code field:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);

RectF qrCodeRect = new RectF(72F, 72F, 288F, 288F);
CPDFQRCodeWidget qrCodeWidget = (CPDFQRCodeWidget) cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_QRCode
    , document);
qrCodeWidget.setRect(qrCodeRect);
qrCodeWidget.setFieldName("QRCode1");
qrCodeWidget.setContent("https://www.compdf.com/");
qrCodeWidget.setErrorCorrectionLevel(CPDFQRCodeWidget.ErrorCorrectionLevel.H);
qrCodeWidget.setQuietZoneModules(6);
qrCodeWidget.updateQRCodeAppearance();
qrCodeWidget.addToAcroFormFields();
```

For error correction levels, quiet zone sizing, and the rules for regenerating the appearance, see 3.3.9 QR Code Form Fields.

3.3.4 Fill Form Fields

ComPDFKit supports programmatically filling form fields in a PDF document.

Since form fields are a special type of annotation, inheriting from the annotation class, the interface for annotations applies to form fields.

The steps to fill form fields using code are as follows:

1. Obtain the page object from CPDFDocument where you want to fill in the form.
2. Retrieve all annotations from the page object.
3. Iterate through all annotations to find the form to be filled.
4. Modify the form field content as needed.

This example shows how to fill form fields:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);

for (int i = 0; i < document.getPageCount(); i++) {
  CPDFPage page = document.pageAtIndex(i);
  for (CPDFAnnotation annotation : page.getAnnotations()) {
    switch (annotation.getType()) {
      case WIDGET:
        CPDFWidget cpdfWidget = (CPDFWidget) annotation;
        switch (cpdfWidget.getWidgetType()) {
          case Widget_TextField:
            CPDFTextWidget textWidget = (CPDFTextWidget) cpdfWidget;
            textWidget.setText("test");
            textWidget.updateAp();
            break;
          case Widget_RadioButton:
            CPDFRadiobuttonWidget radiobuttonWidget = (CPDFRadiobuttonWidget) cpdfWidget;
            radiobuttonWidget.setChecked(true);
            radiobuttonWidget.updateAp();
            break;
          case Widget_ListBox:
            CPDFListboxWidget listBoxWidget = (CPDFListboxWidget) cpdfWidget;
            listBoxWidget.setSelectedIndexes(new int[]{0});
            listBoxWidget.updateAp();
```

```
            break;
          default:
            break;
        }
        break;
      default:
        break;
    }
  }
}
```

3.3.5 Edit Form Fields

Retrieve and edit the appearance and content of form fields.

Note that the properties of different form field types may not be entirely consistent.

The steps to edit a form field are as follows:

1. Obtain the form object to be edited.
2. Modify the form properties.
3. Update the form appearance.

This example shows how to edit form fields:

```
CPDFTextWidget textWidget = (CPDFTextWidget) cpdfWidget;
textWidget.setFillColor(Color.RED.getRGB());
textWidget.updateAp();
```

3.3.6 Delete Form Fields

Since the form fields class inherits from the annotation class, the way to delete form fields is the same as delete annotations.

The steps to delete form fields are as follows:

1. Obtain the page object from CPDFDocument where you want to delete the form field.
2. Delete the form field.

This example shows how to delete form fields:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage page = document.pageAtIndex(0);

List<CPDFAnnotation> annotationList = page.getAnnotations();
annotationList.remove(0);
```

3.3.7 Flatten Forms

Form flattening refers to the process of converting editable form fields into non-editable, static images, or pure text. When flattening form fields, all editable elements in the entire document (including annotations and forms) undergo flattening. Therefore, form flattening is also referred to as document flattening.

This example shows how to flatten forms:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
document.flattenAllPages(CPDFPage.PDFFlattenOption.FLAT_NORMALDISPLAY);
document.save("savePath", isRemoveSecurity);
```

What is Document Flattening?

Document flattening refers to the process of converting editable elements, such as annotations, form fields, or layers, in a PDF document into non-editable, static images, or pure text. The purpose of this process is to lock the final state of the document, eliminating editable elements.

Document flattening is typically applied in the following contexts:

1. Content Protection: Flattening can be used to protect document content, ensuring that the document remains unaltered during distribution or sharing. This is crucial for maintaining document integrity and confidentiality.
2. Form Submission: In form processing, flattening can convert user-filled form fields and annotations into static images for easy transmission, archiving, or printing, while preventing modifications to the form content in subsequent stages.
3. Compatibility and Display: Some PDF readers or browsers may encounter issues with displaying and interacting with PDF documents that contain numerous annotations or layers. Document flattening helps address these compatibility issues, enhancing the visual representation of documents in various environments.
4. File Size Reduction: Flattened documents typically have reduced file sizes since editable elements are converted into static images or text, eliminating the need to store additional data for editing information.

3.3.8 Form Import and Export

ComPDFKit supports exporting form field data to XFDF files or importing form data from XFDF files. This is similar to annotation import/export (see 3.2.8 Import and Export), but specifically for form fields (Widget).

Export Form Data

You can export form field data to an XFDF file by following these steps:

```
CPDFDocument document = new CPDFDocument();
document.open("filePath");
document.exportWidgets("exportPath.xfdf", "tempPath");
```

Import Form Data

You can import form field data from an XFDF file to the document by following these steps:

```
CPDFDocument document = new CPDFDocument();
document.open("filePath");
document.importWidgets("importPath.xfdf", "tempPath");
```

Export Complete Form Properties

In addition to exporting form fill values, you can also export complete Widget properties (including position, style, font, color, and all other configuration information). This is useful when you need to fully backup or migrate form structures.

```
CPDFDocument document = new CPDFDocument();
document.open("filePath");
document.exportCustomWidgets("exportPath.xfdf", "tempPath", null);
```

Import Complete Form Properties

You can import complete Widget properties from an XFDF file to the document:

```
CPDFDocument document = new CPDFDocument();
document.open("filePath");
document.importCustomWidgets("importPath.xfdf", "tempPath");
```

Form Import/Export Method Comparison

Function	Method	Description
Form fill values import/export	importWidgets / exportWidgets	Only import/export form field fill values (user-entered data)

Function	Method	Description
Complete form properties import/export	importCustomWidgets / exportCustomWidgets	Import/export complete Widget properties (position, style, configuration, etc.)

3.3.9 QR Code Form Fields

CPDFQRCodeWidget creates, reads, and modifies QR code fields in a PDF page. The content is supplied as an ordinary Java string and encoded as UTF-8; ComPDFKit then draws the QR code into the field appearance stream (/AP) as vector rectangles. Because nothing is rasterized, the QR code prints and zooms cleanly at any resolution and the document does not carry an extra image resource.

A QR code field is a standard AcroForm Widget annotation. It requires neither JavaScript nor XFA, so the saved document opens in any conforming PDF reader.

Error Correction Levels

The error correction level decides how much of the QR code can be damaged, dirty, or obscured while still being readable. ComPDFKit exposes the four levels defined by the QR code specification through CPDFQRCodeWidget.ErrorCorrectionLevel:

Value	Recoverable damage	Recommended for
ErrorCorrectionLevel.L	About 7%	Long content, clean and controlled scanning conditions.
ErrorCorrectionLevel.M	About 15%	General purpose. This is the default.
ErrorCorrectionLevel.Q	About 25%	Codes that may pick up light wear or smudging.
ErrorCorrectionLevel.H	About 30%	Printed labels, logistics, or anything placed over a background.

A higher level costs capacity: the same field can hold less text, and the generated code usually contains more modules, so each module is drawn smaller within the same rectangle. Raise the level when the code will be printed or handled, and keep it at M when the content is long.

Quiet Zone

The quiet zone is the plain white margin that scanners rely on to locate the code. setQuietZoneModules(int) sets its width in QR modules, not in PDF points, so the margin scales with the code instead of with the page. Valid values are 4 to 64, and the default is 4 — the minimum required by the specification.

Since the quiet zone is drawn inside the field rectangle, increasing it shrinks the code itself. Enlarge it when the QR code sits on a dark or busy background and needs more separation, and leave it at the default otherwise.

Generate and Regenerate the Appearance

updateQRCodeAppearance() encodes the current content and redraws the appearance from the current error correction level, quiet zone, and field rectangle. Calling updateAp() on a QR code field does the same thing, so the field behaves like every other widget in a generic form loop.

Any successful call to setContent(), setErrorCorrectionLevel(), setQuietZoneModules(), or setRect() discards the existing appearance. Always finish a batch of property changes with a single updateQRCodeAppearance() call — there is no need to regenerate after every setter.

Note that setContent() only validates the string. Whether the text actually fits is decided during encoding, so updateQRCodeAppearance() is the call that reports a content-too-long failure.

Read an Existing QR Code

Open the document, walk the annotations of a page, and pick out the QR code fields. Both a type check and the widget type enumeration work:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage page = document.pageAtIndex(0);

for (CPDFAnnotation annotation : page.getAnnotations()) {
  if (annotation instanceof CPDFQRCodeWidget) {
    CPDFQRCodeWidget qrCodeWidget = (CPDFQRCodeWidget) annotation;
    String content = qrCodeWidget.getContent();
    CPDFQRCodeWidget.ErrorCorrectionLevel level = qrCodeWidget.getErrorCorrectionLevel();
    int quietZone = qrCodeWidget.getQuietZoneModules();
    boolean hasAppearance = qrCodeWidget.hasAp();
  }
}
```

getContent() returns null when the widget is invalid; getErrorCorrectionLevel() and getQuietZoneModules() fall back to the defaults M and 4 when the document carries no valid setting.

Update an Existing QR Code

Change the properties you need and regenerate the appearance once at the end:

```
qrCodeWidget.setContent("https://www.compdf.com/contact-us");
qrCodeWidget.setErrorCorrectionLevel(CPDFQRCodeWidget.ErrorCorrectionLevel.Q);
// setContent and setErrorCorrectionLevel both invalidate the old appearance.
qrCodeWidget.updateQRCodeAppearance();
```

Delete a QR Code

A QR code field is deleted the same way as any other form field, through the annotation interface:

```
qrCodeWidget.removeFromPage();
```

The object must not be used again after it has been removed.

API Summary

Method	Description
setContent(String content)	Sets the text to encode. Fails on null or empty content.
getContent()	Returns the encoded text, or null if the widget is invalid.
setErrorCorrectionLevel(ErrorCorrectionLevel level)	Sets the error correction level. The default is M.
getErrorCorrectionLevel()	Returns the current level, or M if none is set.
setQuietZoneModules(int modules)	Sets the white margin in QR modules, from 4 to 64.
getQuietZoneModules()	Returns the quiet zone width, or 4 if none is set.
updateQRCodeAppearance()	Encodes the content and redraws the vector appearance.
addToAcroFormFields()	Registers the field in the document AcroForm fields array.
setRect(RectF rect)	Sets the position on the page. Inherited, invalidates the appearance.
setFieldName(String name)	Sets the AcroForm field name. Inherited.
hasAp()	Reports whether an appearance stream is currently present. Inherited.
removeFromPage()	Removes the field from the page. Inherited.

Error Handling

Every setter and generator returns a boolean, so a failure can be traced to the exact step:

Step	Common cause of failure
addFormWidget(Widget_QRCode, document)	The page is invalid, or the license does not include form permissions.
setRect()	The widget is invalid.
setContent()	The content is null or empty.
setErrorCorrectionLevel()	The level is null.
setQuietZoneModules()	The value is outside [4, 64].
updateQRCodeAppearance()	The content is empty or exceeds the capacity of the code, or the rectangle has a width or height of zero.
addToAcroFormFields()	The document or its AcroForm is in an invalid state.

If a step fails and the new field is no longer wanted, call removeFromPage() to undo the creation before saving.

Notes and Limitations

- The API generates QR codes only. It does not recognize or decode QR codes from an existing page or image.
- The content must be a non-empty string without an embedded NUL character.
- The usable content length depends on both the character set and the error correction level; a higher level leaves room for less text.
- A square rectangle is recommended. A non-square rectangle is accepted — the code is scaled to the shorter side and centred — but it wastes space.
- After changing content, error correction level, quiet zone, or rectangle, the appearance must be regenerated before saving, otherwise the field is written without a visible QR code.
- Call addToAcroFormFields() once the field is configured so that it is saved as a standard form field and stays compatible with common PDF readers.
- Flattening the document turns the QR code into static page content, which is a convenient way to lock it before distribution. See [3.3.7 Flatten Forms](#3.3.7 Flatten Forms).

3.4 Document Editor

3.4.1 Overview

The document editing functionality offers a range of capabilities to manipulate pages, allowing users to control the document structure and adjust the layout and formatting, ensuring that the document content is presented accurately and in a well-organized manner.

Benefits of ComPDFKit Document Editor

- Insertion or Deletion of Pages: Insert or delete pages within the document to meet specific layout requirements.
- Document Merging and Splitting: Combine multiple documents or pages into a new document, or split a large document into smaller ones.
- Structural Adjustments: Adjust the sequence or rotate the orientation of pages to meet specific display or printing needs.

• Multi-Document Collaboration: Extract pages from one document and insert them into another, facilitating collaboration and content integration.

Guides for Document Editor

• [Insert Pages](#3.4.2 Insert Pages)

Insert blank pages, images, or pages from another document into the target document.

• [Split Pages](#3.4.3 Split Pages)

Divide a portion of a multi-page document into independent documents.

• [Merge Pages](#3.4.4 Merge Pages)

Combine pages from multiple documents into a single document.

• [Delete Pages](#3.4.5 Delete Pages)

Remove pages from the document.

• [Rotate Pages](#3.4.6 Rotate Pages)

Rotate pages within a PDF document.

• [Replace Pages](#3.4.7 Replace Pages)

Replace specified pages in the target document with pages from another document.

• [Extract Pages](#3.5.8 Extract Pages)

Extract pages from the document.

3.4.2 Insert Pages

Insert a blank page or pages from other PDFs into the target document.

Insert Blank Pages

This example shows how to insert a blank page:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
int pageIndex = 1; // Insert after the first page.
float pageWidth = 595F;
float pageHeight = 842F;
document.insertBlankPage(pageIndex, pageWidth, pageHeight);
```

Insert Pages from other PDFs

This example shows how to insert pages from other PDFs:

```
CPDFDocument document2 = new CPDFDocument();
document2.open(pdfPath);
document.importPages(document2, new int[]{0}, 1);
```

3.4.3 Split Pages

The steps to split pages are as follows:

1. Create a new CPDFDocument object for each part to be split.
2. Add the portions of the target document that need to be split to the newly created CPDFDocument objects.

This example shows how to split pages:

```
CPDFDocument documentPart1 = CPDFDocument.createDocument();
// Split the first and second pages into separate sections.
documentPart1.importPages(document, new int[]{0,1}, 0);

CPDFDocument documentPart2 = CPDFDocument.createDocument();
// Split the third, fourth, and fifth pages into the second section.
documentPart2.importPages(document, new int[]{2,3,4}, 0);
```

3.4.4 Merge Pages

The steps to merge pages are as follows:

1. Create a blank PDF document.
2. Open the PDF document containing the pages to be merged.
3. Select all the pages to be merged and combine them into the same document.

This example shows how to merge pages:

```
CPDFDocument documentForMerge = CPDFDocument.createDocument();

CPDFDocument document1 = new CPDFDocument();
document1.open(pdfFile);

CPDFDocument document2 = new CPDFDocument();
document2.open(pdfFile);

documentForMerge.importPages(document1, 0, document1.getPageCount() - 1, document.getPageCount());
documentForMerge.importPages(document2, 0, document2.getPageCount() - 1, document.getPageCount());
```

3.4.5 Delete Pages

This example shows how to delete pages:

```
document.removePages(new int[]{0});
```

3.4.6 Rotate Pages

This example shows how to rotate pages:

```
// Rotate the first page 90 degrees clockwise.
document.pageAtIndex(0).setRotation(90);
```

3.4.7 Replace Pages

The steps to replace pages are as follows:

1. Remove the pages in the target file that need to be replaced.
2. Insert the replacement pages into the location where the original document was deleted.

This example shows how to replace pages:

```
// Remove the first page from the document.
document.removePages(new int[]{0});
CPDFDocument document2 = new CPDFDocument();
document2.open(pdfPath);
// Insert the first page of another document into the original document's first-page position to complete the replacement.
document.importPages(document2, new int[]{0}, 0);
```

3.4.8 Extract Pages

This example shows how to extract pages:

```
CPDFDocument extractDocument = CPDFDocument.createDocument();
// Extract the first page of the original document and save it in a new document.
extractDocument.importPages(document, new int[]{0}, 0);
```

3.4.9 Scanned Document Detection

ComPDFKit supports detecting whether a PDF document is a scanned document (image document). Scanned documents are typically generated by scanners, where page content is stored as images rather than selectable text. ComPDFKit provides two levels of scanned document detection: document level and page level.

Detect Scanned Document (Document Level)

Use the CPDFDocument.isImageDoc() method to determine whether the entire PDF document is a scanned document. Here is an example:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");
bool isScannedDocument = document.isImageDoc(); // Returns true if it is a scanned document
if (isScannedDocument)
{
    // This document is a scanned document, may need OCR processing to extract text
}
```

Detect Scanned Page (Page Level)

Use the CPDFPage.isImagePage() method to determine whether a single page is a scanned page (image page). This method is suitable for mixed documents where some pages are scanned images and some pages are regular text. Here is an example:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");
for (int i = 0; i < document.PageCount; i++)
{
    CPDFPage page = document.PageAtIndex(i);
    bool isImagePage = page.isImagePage();// Returns true if the page is a scanned/image page
    if (isImagePage)
    {
        // This page is a scanned page, may need OCR processing to extract text
        System.out.println("page " + i + " is a scanned page");
    }
}
```

Comparison of Two Detection Methods

Method	Level	Use Case
CPDFDocument.IsImageDoc()	Document level	Determine if the entire document is a scanned document, suitable for quick document type filtering
CPDFPage.IsImagePage()	Page level	Determine if a single page is a scanned page, suitable for processing mixed documents or page-by-page analysis

What is a Scanned Document?

A scanned document is a PDF file created by converting a paper document to an electronic document using a scanner or camera. The characteristic of such documents is that page content is stored as images, making it impossible to directly select or search text within. Identifying scanned documents helps determine whether OCR (Optical Character Recognition) processing is needed.

3.5 Security

3.5.1 Overview

The security module provides features including password protection, permission settings, Bates coding, background, and page header and footer functionalities. Document security is guaranteed by managing document passwords and permissions, and by adding logos and copyright information.

Benefits of ComPDFKit Security

- Access Control: Restrict sensitive permissions such as access, copying, or printing by configuring security permissions associated with the document.
- Password Management: Create, modify, or remove document passwords and permissions.
- Encryption Standards: Support for standard PDF security procedures (40 and 128-bit RC4 encryption) as well as 128 and 256-bit AES (Advanced Encryption Standard) encryption.

Guides for Security

- [PDF Permissions](#3.5.2 PDF Permissions)

  By managing document passwords and permission settings, unauthorized access is prevented, and control over user operational permissions for the document is maintained.

- [Background](#3.4.3 Background)

  Setting an image or color background not only enhances the document's visual appeal but also safeguards its privacy, preventing unauthorized copying or distribution.

- [Header and Footer](#3.4.4 Header and Footer)

  Enhance document readability and professionalism by adding marks in the header and footer. Incorporate information such as titles, page numbers, and brand identifiers to facilitate document navigation and recognition.

- [Bates Numbers](#3.4.5 Bates Numbers)

  Insert Bates numbers in the document's header, footer, or other designated locations. Assign unique identification numbers to each page, facilitating document tracking, organization, and legal references.

3.5.2 PDF Permissions

PDF Permissions are employed to ensure the security of PDF documents, offering encryption, document permissions, decryption, and password removal features. This ensures users have secure control and effective management over the document.

Encrypt

Encrypt function consists of two parts: User Password and Owner Password.

The User Password is utilized to open the document, ensuring that only authorized users can access its content. When a user password is set, it typically restricts certain document permissions such as modification, copying, or printing. On the other hand, the Owner Password not only opens the document but also unlocks all restricted permissions, allowing users to modify, copy, or print the document. The dual-password system aims to provide a more flexible and secure approach to document access and management.

ComPDFKit offers a variety of encryption algorithms and permission settings. Depending on your requirements, you can use the appropriate algorithm and configure custom permissions to safeguard your document.

The steps to encrypt are as follows:

1. Set distinct user passwords and owner passwords.

2. Create a permissions information class.

3. Encrypt the document using the user password, owner password, permission information, and the chosen algorithm.

4. Save the document

This sample shows how to encrypt a document:

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
// Set user password.
document.setUserPassword(user_password);
// Set owner password.
document.setOwnerPassword(owner_password);
// Set permission info.
CPDFDocumentPermissionInfo info = document.getPermissionsInfo();
info.setAllowsPrinting(false);
info.setAllowsCopying(true);
document.setPermissionsInfo(info);
// Save document.
document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
```

PDF Permissions

In the PDF specification, there is support for configuring various permissions for a document. By configuring these permissions, it is possible to restrict users to perform only the expected actions.

The PDF specification defines the following permissions:

- Print: Allows printing of the document.
- High-Quality Print: Permits high-fidelity printing of the document.
- Copy: Enables copying of the document content.
- Modify Document: Allows modification of the document content, excluding document properties.
- Assemble Document: Permits insertion, deletion, and rotation of pages.
- Annotate: Enables the creation or modification of document annotations, including form field entries.
- Form Field Input: Allows modification of form field entries, even if document annotations are not editable.

The steps to view document permissions are as follows:

1. Retrieve document permission information.
2. Use the document permission information to view specific permissions.

This example shows how to view document permissions:

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
// get permission info.
CPDFDocumentPermissionInfo info = document.getPermissionsInfo();
System.out.println("Allows Printing:" + info.isAllowsPrinting());
System.out.println("Allows Copying:" + info.isAllowsCopying());
```

Decrypt

Accessing a password-protected PDF document requires entering the password. Different levels of passwords provide varying levels of permission.

The steps for decryption are as follows:

1. When opening the document, check if it is encrypted.

2. For encrypted documents, entering either the user password or the owner password allows the document to be opened.

This example shows how to decrypt a document:

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
CPDFDocument.PDFDocumentError error = document.open(pdfPath);
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorPassword) {
    // Password required.
    document.open(pdfFath, password);
}
```

Remove passwords

Removing passwords means deleting the owner passwords and user passwords from a document and saving it as a new document, which will no longer require a password to open and will have all permissions available by default.

The steps to remove passwords are as follows:

1. Unlock the document to obtain all permissions.
2. Save the unlocked document.

This example shows how to remove passwords:

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath, password);
// Remove password.
try {
  document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveRemoveSecurity);
} catch (CPDFDocumentException e) {
  e.printStackTrace();
}
```

3.5.3 Background

The background refers to the underlying layer or pattern on the document pages, used to present the fundamental visual effect of the document. Adding a background can alter the document's appearance, making it more personalized or professional. It can be used to emphasize a brand, protect copyright, or enhance the reading experience of the document.

In a PDF document, only one background can exist, and adding a new background to pages containing an existing background will overwrite the old background.

Set Color Background

The steps to set a color background are as follows:

1. Obtain the document's background object.
2. Set the background type to color.
3. Configure the properties of the background.
4. Update the background of the document.

This example shows how to set the color background:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");

CPDFBackground background = document.getBackground();
background.setType(CPDFBackground.Type.Color);
background.setColor(ColorUtils.parseColor(1f, 1f, 0f, 0f)); // RGBA
background.setOpacity(0.5f);
background.setRotation(45f);
```

```
background.setScale(1.0f);
background.setHorizalign(CPDFBackground.CPDFBackgroundAlign.Center);
background.setVertalign(CPDFBackground.CPDFBackgroundAlign.Center);
background.setXOffset(0f);
background.setYOffset(0f);
background.setPages("0,1,2");
background.setAllowsView(true);
background.setAllowsPrint(true);
background.update();

document.save(outputPath, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
background.release();
document.close();
```

Set Image Background

The steps to set an image background are as follows:

1. Obtain the document background object.
2. Set the background type to an image.
3. Specify the background properties.
4. Update the background on the document.

This example shows how to set the image background:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");

CPDFBackground background = document.getBackground();
background.setType(CPDFBackground.Type.Image);
background.setImage("ComPDFKit.png", "", null);
background.setOpacity(1.0f);
background.setScale(1.0f);
background.setHorizalign(CPDFBackground.CPDFBackgroundAlign.Center);
background.setVertalign(CPDFBackground.CPDFBackgroundAlign.Center);
background.setPages("0,1,2,3,4");
background.setAllowsView(true);
background.setAllowsPrint(true);
background.update();

document.save(outputPath, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
background.release();
document.close();
```

Remove Background

The steps to remove the background are as follows:

1. Obtain the document background object.
2. Delete the document background.

This example shows how to remove background:

```
CPDFDocument document = new CPDFDocument();
document.open(inputPath);
CPDFBackground background = document.getBackground();

System.out.println("isValid: " + background.isValid());
System.out.println("Type: " + background.getType());
System.out.println("Scale: " + background.getScale());
System.out.println("Rotation: " + background.getRotation());
System.out.println("Horizalign: " + background.getHorizalign());
System.out.println("Vertalign: " + background.getVertalign());
System.out.println("Pages: " + background.getPages());

background.clear();
document.save(outputPath, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
background.release();
document.close();
```

3.5.4 Header and Footer

Header and Footer refer to annotations added at the top and bottom of a document, typically containing information such as titles, page numbers, and brand identification. By including headers and footers in a document, it becomes easier to navigate and identify, thereby enhancing the document's readability and professionalism.

In a PDF document, only one header and footer can exist, and adding a new header and footer will overwrite the old header and footer.

Add Header and Footer

The steps to add headers and footers are as follows:

1. Retrieve the header and footer objects within the document.
2. Specify the pages where headers and footers should be added.
3. Set attributes such as color and font size for the headers and footers.
4. Update the headers and footers on the pages.

This example shows how to add header and footer:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");

CPDFHeaderFooter headerFooter = document.getHeaderFooter();

// Top middle (index 1)
headerFooter.setText(1, "Header - <<1>>");
headerFooter.setFontName(1, "Helvetica");
headerFooter.setFontSize(1, 14f);
headerFooter.setTextColor(1, 0xFF000000);

// Bottom middle (index 4)
headerFooter.setText(4, "Page <<1>>");
headerFooter.setFontName(4, "Helvetica");
headerFooter.setFontSize(4, 12f);
headerFooter.setTextColor(4, 0xFF000000);

headerFooter.setMargin(20f, 20f, 20f, 20f);
headerFooter.setPageOffset(1);
headerFooter.setPages("0,1,2,3,4");
headerFooter.setRules(0);

headerFooter.update();

document.save(outputPath, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
headerFooter.release();
document.close();
```

Header and Footer Regular Expression Explanation

Headers and footers support format-specific regular expressions that take effect when headerFooter.setRules(1) is set, in the format: <<\d+,\d+>>|<<\d+>>|<\d+,>>

- **<<i>>** : iis the starting value of the page number.
- **<<i,f>>**:iis the starting value of the page number, andfis the number of digits in the page number, if the actual page number is not enough, it will be automatically filled with 0 in front.

eg: When text is set to "**<<**1,2**>>** page", the text displayed on the first page is "01 page".

Remove Header and Footer

Steps to Remove headers and footers:

1. Retrieve the header and footer objects from the document.
2. Remove the headers and footers.

This example shows how to remove the header and footer:

```
CPDFDocument document = new CPDFDocument();
document.open(inputPath);

CPDFHeaderFooter headerFooter = document.getHeaderFooter();
System.out.println("isValid: " + headerFooter.isValid());

headerFooter.clear();
document.save(clearPath, CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
headerFooter.release();
document.close();
```

3.5.5 Bates Numbers

ComPDFKit provides a comprehensive API for adding, editing, and deleting Bates numbers in PDF documents, facilitating the management of user security information.

In a PDF document, only one Bates number can exist, and adding a new Bates number will overwrite the old Bates numbers.

Add Bates numbers

The steps to add Bates numbers are as follows:

1. Retrieve the Bates numbers object from the document.
2. Set the properties for the Bates numbers to be added.
3. Update the Bates numbers in the document.

This example shows how to add the Bates number:

```
CPDFDocument document = CPDFDocument.InitWithFilePath("filePath");

CPDFBates bates = document.GetBates();
bates.SetText(0, @"<<#3#5#Prefix-#-Suffix>>");
byte[] color = { 255, 0, 0 };
bates.SetTextColor(0, color);
bates.SetFontSize(0,14);
bates.SetPages("0-" + (document.PageCount - 1));
bates.Update();
```

Bates numbers Regular Expression Explanation

The regular expression for Bates numbers supports a specific format: **<<**#\d+#\d+#\w+#\w+**>>**

- The first # is followed by the minimum number of digits to display for the page number. If the page number has fewer digits, leading zeros are added.
- The second # is followed by the starting value of the page number.
- The third # is followed by the prefix for the header/footer.
- The fourth # is followed by the suffix for the header/footer.

For example: When the text is set to "<<#3#1#ab#cd>>," the text displayed on the first page will be "ab001cd."

Remove Bates numbers

The steps to remove Bates numbers are as follows:

1. Retrieve the Bates numbers object from the document.
2. Remove the Bates numbers.

This example shows how to remove the Bates numbers:

```
CPDFHeaderFooter headerFooter = document.GetHeaderFooter();
headerFooter.Clear();
```

3.6 Redaction

3.6.1 Overview

Redaction is the process of removing visible text or images from a document, often permanently deleting or overwriting sensitive information and completely removing it from the PDF metadata. Revised content is often replaced with black bars or other visually prominent markings to indicate that the information has been redacted.

Redaction is commonly employed to safeguard privacy, comply with legal requirements, or ensure that sensitive content is not inadvertently disclosed. This process facilitates secure distribution to audiences such as courts, patent and government agencies, media, clients, suppliers, or any other restricted-access entities.

Benefits of ComPDFKit Redaction

- Text Encryption: Sensitive information is rendered unreadable by overlaying it with colored blocks or specific characters.
- Image Encryption: Protect confidential information by hiding or removing image areas with black boxes or special characters.
- Permanent Alteration: Redaction is a permanent process, ensuring that sensitive information cannot be recovered.
- Metadata Purge: In addition to removing visible content, any hidden metadata in the PDF that may contain sensitive information is also eliminated.

Guides for Redaction

- [Redact PDFs](#3.6.2 Redact PDFs)

  Redact specific information in PDF documents.

3.6.2 Redact PDFs

The steps to redact PDFs are as follows:

1. Create Redaction Annotations: A user applies to redact annotations that specify the pieces or regions of content that should be removed. This step marks the regions where redaction changes will be applied. These redaction annotations can be set, moved, or deleted before applying redaction changes, without removing content from the document.

2. Apply Redaction Changes: After marking the areas for complete content removal, apply redaction changes. The content within the regions of the redaction annotations will be irreversibly deleted.

Through these two steps, you can thoroughly remove sensitive data from the document.

Create Redaction Annotations

You can use the CPDFRedactAnnotation class to create redaction annotations. Utilize the setRect methods to define the areas that should be covered by the redaction annotations.

In addition, the appearance of the ciphertext can be customized at this stage. Note that once redaction annotations have been applied, their appearance cannot be changed. This is because the redaction annotations will become non-editable, non-modifiable, static content instead of an interactive ciphertext annotation after it has been applied.

This sample shows how to create a redaction annotation:

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage pdfPage = document.pageAtIndex(0);
CPDFRedactAnnotation redactAnnotation = (CPDFRedactAnnotation) pdfPage.addAnnot(CPDFAnnotation.Type.REDACT,document);
RectF insertRect = new RectF(0, 0, 100, 100);
redactAnnotation.setRect(insertRect);
CPDFTextAttribute textAttribute = new CPDFTextAttribute("Helvetica", 14, Color.YELLOW.getRGB());
redactAnnotation.setTextDa(textAttribute);
redactAnnotation.setFillColor(Color.RED.getRGB());
redactAnnotation.setOverLayText("REDACTED");
redactAnnotation.updateAp();
redactAnnotation.applyRedaction();
```

Apply Redaction Changes

ComPDFKit SDK ensures that if text, images, or vector graphics are included in the region marked by a redaction annotation, the corresponding image or path data in that portion will be completely removed and cannot be restored.

This sample shows how to apply redaction changes:

```
redactAnnotation.applyRedactions();
```

3.7 Watermark

3.7.1 Overview

A watermark is a mark placed on a PDF document, typically consisting of semi-transparent text or image elements added to the PDF. By adding a watermark, document source and copyright information can be displayed without disrupting readability.

Benefits of ComPDFKit PDF Watermark SDK

- Copyright Notice: Display document source and copyright information through a watermark to prevent screenshots or unauthorized use.
- Branding: Customize the appearance of the watermark to showcase a brand logo or other predefined content.

Guides for Watermark

- [Add Text Watermark](#3.7.2 Add Text Watermark)

  Create a text watermark with custom content and appearance.

- [Add Image Watermark](#3.7.3 Add Image Watermark)

  Generate an image watermark using a custom image.

- [Delete Watermark](#3.7.4 Delete Watermark)

  Remove the watermark attached to the PDF document.

3.7.2 Add Text Watermark

The steps to add a text watermark are as follows:

1. Initialize a CPDFWatermark object, specifying the watermark type as text.
2. Set the properties required for the text watermark, including content, font, color, and font size.
3. Set the general properties for the watermark.
4. Create the watermark in the document.

This example shows how to add a text watermark:

```
CPDFWatermark watermark = (CPDFWatermark) document.createWatermark(CPDFWatermark.Type.WATERMARK_TYPE_TEXT);
watermark.setText("Hello World");// Text for the watermark (not applicable for image watermarks).
watermark.setFontName("Helvetica");// Text font for the watermark (not applicable for image watermarks).
watermark.setTextRGBColor(Color.decode(color).getRGB());// Text color for the watermark (not applicable for image watermarks).
watermark.setFontSize(30);// Default font: Helvetica 24.
watermark.setOpacity(0.5f);// Opacity of the watermark, ranging from 0 to 1, with a default of 1.
watermark.setFront(true);// Position the watermark in front of the content.
watermark.setVertalign(CPDFWatermark.Vertalign.WATERMARK_VERTALIGN_CENTER);// Vertical alignment of the watermark.
watermark.setHorizalign(CPDFWatermark.Horizalign.WATERMARK_HORIZALIGN_CENTER);// Horizontal alignment of the watermark.
watermark.setVertOffset(0);// Translation offset relative to the vertical position. Positive values move downward, while negative values
    move upward.
watermark.setHorizOffset(0);// Translation offset relative to the horizontal position. Positive values move to the right, while negative
    values move to the left.
watermark.setScale(1.3f);// Scaling factor for the watermark, with a default value of 1. If it is an image watermark, 1 represents the
    original size of the image. If it is a text watermark, 1 represents the `textFont` font size.
watermark.setPages("3,4,5");// Set the watermark for pages 3, 4, and 5.
watermark.setFullScreen(true);// Enable watermark tiling (not applicable for image watermarks).
watermark.setHorizontalSpacing(100);// Set the horizontal spacing for tiled watermarks.
watermark.setVerticalSpacing(100);// Set the vertical spacing for tiled watermarks.
watermark.update();
watermark.release();
```

3.7.3 Add Image Watermark

The steps to add an image watermark are as follows:

1. Initialize a CPDFWatermark object, specifying the watermark type as an image.
2. Create a Bitmap based on the image file, setting the image source and scaling for the image watermark.
3. Set the general properties for the watermark.
4. Create the watermark in the document.

This example shows how to add an image watermark:

```
CPDFWatermark watermark = (CPDFWatermark) document.createWatermark(CPDFWatermark.Type.WATERMARK_TYPE_IMG);
watermark.setImage(bitmap, 100, 200);// Set the image for the watermark (not applicable for text watermarks).
watermark.setOpacity(0.5f);// Watermark opacity, ranging from 0 to 1, with a default of 1.
watermark.setFront(true);// Position the watermark in front of the content.
watermark.setVertalign(CPDFWatermark.Vertalign.WATERMARK_VERTALIGN_CENTER);// Vertical alignment of the watermark.
watermark.setHorizalign(CPDFWatermark.Horizalign.WATERMARK_HORIZALIGN_CENTER);// Horizontal alignment of the watermark.
watermark.setVertOffset(0);// Translation offset relative to the vertical position. Positive values move downward, while negative values
    move upward.
watermark.setHorizOffset(0);// Translation offset relative to the horizontal position. Positive values move to the right, while negative
    values move to the left.
watermark.setScale(1.3f);// Scaling factor for the watermark, with a default value of 1. If it is an image watermark, 1 represents the
    original size of the image. If it is a text watermark, 1 represents the `textFont` font size.
watermark.setPages("3,4,5");// Set the watermark for pages 3, 4, and 5.
watermark.setFullScreen(true);// Enable watermark tiling (not applicable for image watermarks).
watermark.setHorizontalSpacing(100);// Set the horizontal spacing for tiled watermarks.
watermark.setVerticalSpacing(100);// Set the vertical spacing for tiled watermarks.
watermark.update();
watermark.release();
```

3.7.4 Delete Watermark

To delete watermarks, follow these steps:

Call releaseAllWatermarks() on the CPDFDocument object to remove all watermarks from the document.

This example shows how to delete the watermark:

```
//remove all watermarks from the document.
document.releaseAllWatermarks();
```

3.8 Conversion

3.8.1 PDF Standard Format Conversion

ComPDFKit PDF SDK supports converting PDF documents to various PDF standard formats, including PDF/A (for long-term archiving), PDF/X (for printing), PDF/E (for engineering documents), PDF/UA (for accessibility), and PDF/VT (for variable data printing).

Convert to Standard PDF Format

Use the WriteStandardPDFToFilePath method to convert an existing PDF file to a specified PDF standard format. This method analyzes the content of the existing PDF file and makes a series of modifications to generate a document compliant with the target standard.

During the conversion process, features unsuitable for the target standard (such as encryption, outdated compression schemes, missing fonts, or device-dependent colors) will be replaced with standard-compliant equivalents. Since the conversion process applies only the necessary changes to the source file, minimal information loss occurs.

Method Signature:

```
public bool WriteStandardPDFToFilePath(
    PDFDocumentType standardType,  // PDF standard type
    String iccFilePath,            // ICC color profile path (optional, leave empty for default)
    String saveFilePath,           // Output file save path
    CPDFUAConfig uaConfig  // PDF/UA configuration (only required for PDF/UA format)
)
```

Parameters:

Parameter	Type	Description
standardType	PDFDocumentType	Target PDF standard type, containing format, version number, and subtype
iccFilePath	string	ICC color profile path. If the file does not exist, the default configuration file will be automatically selected based on the standard type
saveFilePath	string	Save path for the converted document
uaConfig	CPDFUAConfig	PDF/UA configuration (optional), used to set document title and language

Example - Convert to PDF/A-1a:

```
CPDFDocument doc1 = new CPDFDocument();
doc1.open("test.pdf");
boolean ok1 = doc1.WriteStandardPDFToFilePath(
        CPDFDocument.PDFDocumentType.PDFTypeA1a,
        "sRGB2014.icc",
        "convpdfa1a.pdf");
doc1.close();
```

Example - Convert to PDF/UA:

```
String filePath = "test.pdf";
CPDFDocument document = new CPDFDocument();
document.open(filePath);

CPDFUAConfig ua = new CPDFUAConfig();
ua.title = "document";
ua.language = CPDFUAConfig.UALanguage.cs_CZ;

String iccPath = "sRGB2014.icc";
String outPath = "convpdfUA_config.pdf";

boolean ok = document.WriteStandardPDFToFilePath(
        CPDFDocument.PDFDocumentType.PDFTypeUA1, iccPath, outPath, ua);
document.close();
System.out.println("convertPDFUA with config: " + ok);
```

Detect PDF Standard Type

Use the GetPDFStandardType method to detect whether a document currently conforms to a PDF standard format.

Method Signature:

```
public PDFStandardType GetPDFStandardType()
```

Return Value:

Returns a PDFStandardType struct containing the following fields:

Field	Type	Description
Format	CPDFStandardFormat	PDF standard format (StandardPDF, StandardPDFA, StandardPDFX, etc.)
TypeNum	int	Version number (e.g., TypeNum is 1 for PDF/A-1)
Type	CPDFPartType	Subtype (PDF_a, PDF_b, PDF_u, or None)

Example:

```
CPDFDocument doc1 = new CPDFDocument();
doc1.open("test.pdf");
CPDFStandardType standardType = document.GetPDFStandardType();
switch (standardType.Format)
{
   case CPDFStandardFormat.StandardPDF:
      System.out.println("common PDF document");
      break;
   case CPDFStandardFormat.StandardPDFA:
      System.out.println("PDF/A-"+standardType.TypeNum + GetPartTypeSuffix(standardType.Type) + "document");
      break;
   case CPDFStandardFormat.StandardPDFX:
   System.out.println("PDF/X-"+standardType.TypeNum + "document");
      break;
   case CPDFStandardFormat.StandardPDFUA:
      System.out.println("PDF/UA-" + standardType.TypeNum + "document");
      break;
   default:
```

```
        System.out.println("Other standard format: " + standardType.Format);
        break;
}

// Helper method: get subtype suffix
String GetPartTypeSuffix(CPDFPartType partType)
{
   if (partType == PDFPartType.PDF_a) {
     return "a";
   } else if (partType == PDFPartType.PDF_b) {
     return "b";
   } else if (partType == PDFPartType.PDF_u) {
     return "u";
   } else return "";
}
```

3.8.2 PDF Standard Format Details

CPDFStandardFormat Enumeration

Enum Value	Description
StandardPDF	Standard PDF format (no specific standard)
StandardPDFA	PDF/A format, for long-term archiving
StandardPDFX	PDF/X format, for print exchange
StandardPDFE	PDF/E format, for engineering documents
StandardPDFUA	PDF/UA format, for universal accessibility
StandardPDFVT	PDF/VT format, for variable data printing

CPDFPartType Enumeration

Enum Value	Description
None	No subtype
PDF_a	Type a (full conformance, includes structural information)
PDF_b	Type b (basic conformance)
PDF_u	Type u (Unicode conformance)

CPDFUALanguage Enumeration (Partial)

Enum Value	Description
unspecified	Unspecified
zh_CN	Simplified Chinese (Mainland China)
zh_TW	Traditional Chinese (Taiwan)
en_US	English (United States)
en_GB	English (United Kingdom)
ja_JP	Japanese (Japan)
ko_KR	Korean (South Korea)
de_DE	German (Germany)
fr_FR	French (France)

3.8.3 PDF Standard Introduction

PDF standards are a series of PDF format specifications defined by the International Organization for Standardization (ISO), each optimized for specific use cases. ComPDFKit supports conversion to the following PDF standard formats.

PDF/A (Archiving Standard)

What is PDF/A?

PDF/A (Portable Document Format Archival) is a PDF file format standard specifically designed for the long-term preservation of electronic documents. Its purpose is to ensure that documents maintain readability and accessibility over time, meeting the requirements for archiving and long-term preservation.

PDF/A Versions

Name	Release Date	Standard	Based On	Description
PDF/A-1	2005-09-28	ISO 19005-1	PDF 1.4	Basic archiving format, divided into 1a (full conformance) and 1b (basic conformance)
PDF/A-2	2011-06-20	ISO 19005-2	PDF 1.7	Supports JPEG2000, transparency, layers; adds 2u (Unicode) subtype
PDF/A-3	2012-10-15	ISO 19005-3	PDF 1.7	Allows embedding of arbitrary file formats (e.g., XML, CAD)
PDF/A-4	2020-11	ISO 19005-4	PDF 2.0	Based on the latest PDF 2.0, simplifies conformance levels

PDF/A Subtype Description

Subtype	CPDFPartType	Description
a	PDF_a	Full conformance, includes tag structure information, supports assistive technology access
b	PDF_b	Basic conformance, ensures visual appearance can be reproduced
u	PDF_u	Unicode conformance (PDF/A-2 and above), ensures text is searchable and extractable

Key Features of PDF/A

- Self-Containment: The document contains all necessary resources (fonts, images, etc.), ensuring independent rendering.
- Font Embedding: All fonts must be embedded to prevent display inconsistencies.
- No Encryption: Encryption is not allowed to ensure long-term accessibility.
- Metadata Requirements: Must include XMP metadata.
- Color Management: Must use ICC color profiles or device-independent color spaces.

PDF/X (Print Exchange Standard)

What is PDF/X?

PDF/X (Portable Document Format Exchange) is a PDF standard designed for the printing industry, ensuring reliable document exchange and output across different printing workflows. It eliminates common technical issues in the printing process, such as missing fonts and color discrepancies.

PDF/X Versions

Name	Standard	Primary Use
PDF/X-1a	ISO 15930-1	CMYK and spot color printing, suitable for traditional printing workflows
PDF/X-3	ISO 15930-3	Color-managed printing, allows RGB and Lab color spaces
PDF/X-4	ISO 15930-7	Supports transparency and layers, suitable for modern printing workflows
PDF/X-5	ISO 15930-8	Supports external referenced content (OPI), for partial exchange scenarios

Key Features of PDF/X

- Color Accuracy: Must use CMYK, spot colors, or color spaces with ICC profiles.
- Font Embedding: All fonts must be embedded.
- Bleed and Trim: Supports defining bleed areas (BleedBox) and trim areas (TrimBox).
- No Transparency (X-1a/X-3): Earlier versions require flattened transparency.
- Output Intent: Must include OutputIntent information specifying the target output device or condition.

Use Cases: Commercial printing, magazine publishing, packaging printing.

PDF/E (Engineering Document Standard)

What is PDF/E?

PDF/E (Portable Document Format Engineering) is a PDF standard designed for the engineering field, used for exchanging engineering documents, technical drawings, and 3D models.

PDF/E Versions

Name	Standard	Key Features
PDF/E-1	ISO 24517-1	Based on PDF 1.6, supports 3D content, layers, and annotations
PDF/E-2	ISO 24517-2 (in development)	Plans to support more engineering features

Key Features of PDF/E

- 3D Support: Can embed 3D models in U3D and PRC formats.
- Layer Support: Supports Optional Content Groups (OCG) for viewing different layers.
- Precise Measurement: Supports geospatial coordinates and measurement units.
- Large Format Support: Suitable for large-size pages in engineering drawings.
- Annotations and Markups: Supports markups and annotations in engineering reviews.

Use Cases: CAD drawing exchange, architectural design, manufacturing technical documentation.

PDF/UA (Universal Accessibility Standard)

What is PDF/UA?

PDF/UA (Portable Document Format Universal Accessibility) is a PDF standard designed for universal accessibility, ensuring that people with disabilities (especially visually impaired users) can access PDF content through assistive technologies (such as screen readers).

PDF/UA Versions

Name	Standard	Key Features
PDF/UA-1	ISO 14289-1	Based on PDF 1.7, defines technical requirements for accessible access
PDF/UA-2	ISO 14289-2 (2024)	Based on PDF 2.0, enhanced support for mathematical formulas and annotations

Key Features of PDF/UA

- Tag Structure: Must include a complete tag tree (Tagged PDF) defining the logical structure of content.
- Reading Order: Content must have a correct reading order.
- Alternative Text: Images and non-text content must provide alternative text descriptions.
- Language Identification: Must specify the document language to support correct pronunciation by screen readers.
- Heading Hierarchy: Headings must have a correct hierarchical structure.
- Table Marking: Tables must have correct row/column markings.
- Navigation Support: Supports bookmark and link navigation.

Use Cases: Government documents, educational materials, public service documents, documents required to comply with accessibility regulations.

PDF/VT (Variable Data Printing Standard)

What is PDF/VT?

PDF/VT (Portable Document Format Variable and Transactional) is a PDF standard designed for variable data printing (VDP), used for efficiently processing large-volume print jobs with personalized content, such as bills, direct mail letters, and personalized marketing materials.

PDF/VT Versions

Name	Standard	Key Features
PDF/VT-1	ISO 16612-2	Single-file exchange, contains all resources
PDF/VT-2	ISO 16612-2	Multi-file exchange, supports externally referenced resources

Key Features of PDF/VT

- Document Part Metadata (DPM): Defines the logical structure of the document (e.g., recipients, page groups).
- Resource Reuse: Supports efficient reuse of XObject resources, reducing file size.
- Incremental Processing: Supports streaming processing, suitable for large-volume jobs.
- Based on PDF/X-4: Inherits the printing features of PDF/X-4.
- Variable Content Marking: Identifies static content and variable content areas.

Use Cases: Bill printing, personalized direct mail, bulk certificate generation, variable data marketing materials.

PDF Standard Comparison Summary

Standard	Primary Use	Key Features	Typical Applications
PDF/A	Long-term archiving	Self-contained, no encryption, font embedding	Legal documents, government archives, historical records
PDF/X	Print exchange	Color accuracy, output intent, bleed definition	Commercial printing, magazines, packaging
PDF/E	Engineering documents	3D support, layers, precise measurement	CAD drawings, architectural design
PDF/UA	Universal accessibility	Tag structure, alternative text, reading order	Government documents, educational materials
PDF/VT	Variable printing	Resource reuse, batch processing, personalization	Bills, direct mail, personalized marketing

3.9 Documentation comparison

3.9.1 Introduction

ComPDFKit Watermarking Functions

- [Overlay Comparison](## 3.8.2 Adding a Text Watermark)

Overlay Compare is used to visually compare pages from different documents. It is useful for construction plans, detailed drawings, and other content that requires precise placement.

• [Content Comparison](##3.8.3 Add Image Watermark)

Quickly identify changes by comparing two versions of a PDF file.

3.9.2 Overlay Comparison

To perform an overlay comparison, proceed as follows:

1. Open the two documents being compared.
2. Create the CPDFCompareDrawings object.
3. Set the colour and other properties of the comparison.
4. Call compareDrawings.getDocGenerate() to get the comparison results and save them to the document.

This example shows how to add an overlay comparison:

```
CPDFDocument baseDocument = new CPDFDocument();
baseDocument.open(rootDir + "/TestFiles/comparefile.pdf");
CPDFDocument comDocument = new CPDFDocument();
comDocument.open(rootDir + "/TestFiles/comparefile1.pdf");
printDividingLine();

CPDFCompareDrawings compareDrawings = baseDocument.createCompareDrawings(comDocument);
compareDrawings.setBlendMode(4);
compareDrawings.setColorCompare(Color.decode("#FBBDBF").getRGB());
compareDrawings.setColorCompared(Color.decode("#93B9FD").getRGB());
compareDrawings.setFillAlphaCompare(50);
compareDrawings.setFillAlphaCompared(50);
compareDrawings.compareContent();
CPDFDocument document = compareDrawings.getDocGenerate();
document.save(rootDir + "/out/compareTestOverlay.pdf", CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
```

3.9.3 Content Comparison

The steps to perform a content comparison are as follows:

1. Open the two documents being compared.
2. Create the CPDFCompare object.
3. Set the colour and other properties of the comparison.
4. Call compare.doCompare() to get the comparison results and save them to the document using the generateNewDoc() method.

This example shows how to add an content comparison:

```
CPDFDocument baseDocument = new CPDFDocument();
baseDocument.open(rootDir + "/TestFiles/comparefile.pdf");
CPDFDocument comDocument = new CPDFDocument();
comDocument.open(rootDir + "/TestFiles/comparefile1.pdf");
printDividingLine();

CPDFCompare compare = baseDocument.createCompare(baseDocument, comDocument);
compare.setReplaceResColor(Color.decode("#93B9FD").getRGB());
compare.setInsertResColor(Color.decode("#C0FFEC").getRGB());
CPDFCompareResults result = compare.doCompare(0, 0, CPDFCompare.CompType.COMPAREALL.id, true);
compare.generateNewDoc(rootDir + "/out/compareTest.pdf");
```

3.10 Digital Signatures

3.10.1 Overview

A digital signature is legally binding and can be equivalent to an ink pen signature on paper contracts and other documents.

Unlike electronic signatures, digital signatures have a unique digital ID that identifies the signer's identity. Digital signatures can get information about whether the signature is trustworthy and whether the document has been modified after the signature, thereby ensuring the legal validity of the document.

Benefits of ComPDFKit PDF Digital Signature SDK

- Authentication: Digital signatures can accurately identify the creator and the signer of a document.
- Integrity: Digital signatures allow users to easily verify whether the document's content has been altered after signing.
- Non-Repudiation: When the signature is valid, it can prove the signer's intent to sign, they can't deny that they have signed the document.
- Built-in Certificate Support: Full support for PFX and P12 certificates.
- Custom Appearance: Customize the appearance of signatures through drawn, image, or typed signatures.
- Fast UI Integration: Achieve rapid integration and customization through extensible UI components.

Guides for Digital Signatures

- Create Digital Certificates: Create a PKCS#12 signing identity.
- Create Digital Signatures: Create a field and sign it with a certificate.
- Read Digital Signature Information: Inspect signer and certificate information.
- Verify Digital Certificates: Check certificate trust.
- Verify Digital Signatures: Check integrity, permissions, and trust.
- Trust Certificate: Configure certificates trusted by the application.
- Remove Digital Signatures: Remove signature data and optionally its appearance while retaining the field.
- Invisible Digital Signatures: Sign with a widget rectangle of zero.
- Long-Term Validation and PAdES: Create or prolong evidence and inspect full validation reports.
- AWS KMS Signing: Use the supported KMS key configuration and certificate.
- Two-Phase External Signing: Persist preparation and complete it after the external signer responds.
- Signature and Document Timestamps: Use embedded timestamps, document timestamps, and TSA tokens.

The advanced signature APIs below require an SDK JAR that contains the corresponding classes. Initialize and license the SDK before use. The code examples use these imports as needed:

```
import com.compdfkit.core.android.graphics.RectF;
import com.compdfkit.core.annotation.form.CPDFSignatureWidget;
import com.compdfkit.core.annotation.form.CPDFWidget;
import com.compdfkit.core.document.CPDFDocument;
import com.compdfkit.core.page.CPDFPage;
import com.compdfkit.core.signature.*;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Arrays;
import java.util.Base64;
```

3.10.2 Concepts of Digital Signatures

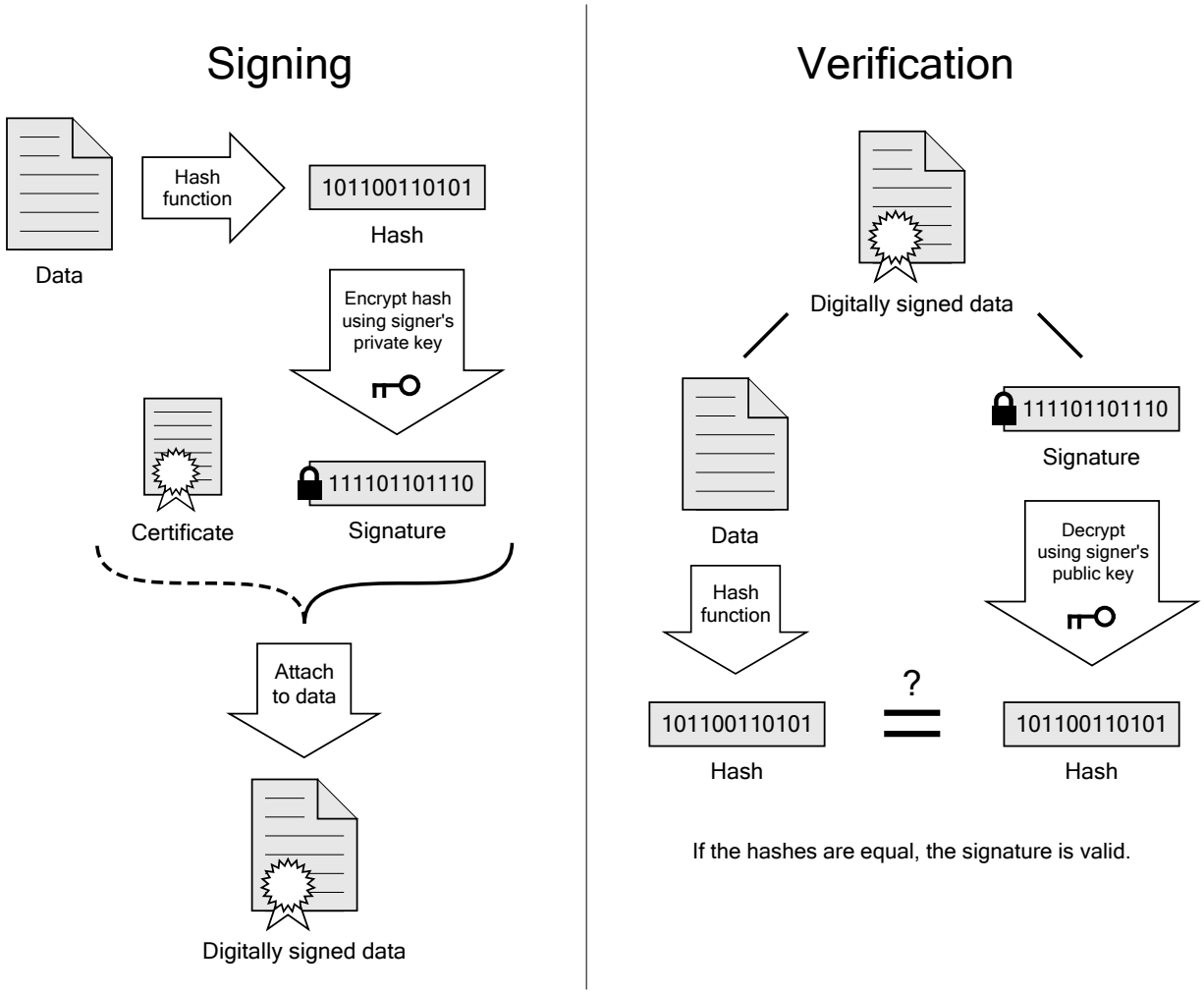
How Digital Signatures Work

Principle of Signature:

A hash value of the data to be encrypted is obtained through a hash function (a unique fingerprint of the data. Any tampering with the data content will result in a different hash). The hash value is encrypted using the signer's private key to obtain the digital signature. The signed data will be generated after attaching the digital signature to the data.

Verification Principle:

Separate the signature from the data, and obtain the hash value of the data through the same hash function used by the signer. Decrypt the hash value using the signer's public key to get the signer's hash value. By comparing the two values, we can confirm whether the file has been tampered with.



Digital Signatures vs Electronic Signatures

An electronic signature is essentially an annotation within a document. Apart from the customizable appearance of the signature, it lacks identifiable information about the creator and cannot verify whether the document has been altered.

However, a digital signature uses complex encryption algorithms to create a unique identifier that is linked to both the document's content and the creator's information. Any modification to the document's content results in a failed digital signature verification, ensuring the uniqueness and legitimacy of the signer's identity.

What Is a Digital Certificate?

A digital certificate is a digital authentication that marks the identity information of the parties in Internet communication. It can be used online to identify the identity of the other party, hence, it is also known as a digital ID.

The format of the digital certificate typically adopts the X.509 international standard. It will generally include the certificate's public key, user information, the validity period of the public key, the name of the certificate authority, the serial number of the digital certificate, and the digital signature of the issuing organization.

Digital certificates provide the transmission of information and data in an encrypted or decrypted form during communication between network users, ensuring the integrity and security of information and data.

Support PKCS12 Certificate

PKCS12 (Or PKCS #12) is one of the family of standards called Public-Key Cryptography Standards (PKCS) published by RSA Laboratories. ComPDFKit supports signing PDFs with PKCS12 files which are with ".p12" or ".pfx" file extensions.

What Is Certificate Chain

A Certificate Chain (Chain of Trust), is an ordered collection of digital certificates used to verify the authenticity and trustworthiness of a digital certificate. Certificate chains are typically employed to establish trust, ensuring that both the public key and the identity of entities are legitimate and trustworthy.

Here are some key concepts within a certificate chain:

- Root Certificate

  The starting point of a certificate chain is the Root Certificate. Root certificates are top-level certificates issued by trusted Certificate Authorities (CAs) and are often built into operating systems or applications. These root certificates serve as the foundation of trust because they are considered inherently trustworthy.

- Intermediate Certificates

  Intermediate certificates, also known as issuer certificates or sub-certificates, are issued by root certificate authorities and are used to issue certificates for end entities. Intermediate certificates form an intermediate link within the certificate chain.

- End Entity Certificate

  An end entity certificate is the certificate of the subject of a digital signature (typically an individual, server, or device). These certificates are issued by intermediate certificate authorities and contain the public key and relevant identity information.

- Trust Establishment

  Trust is established through the certificate chain, passing trust from the root certificate to the end entity certificate. If the root certificate is trusted, then the end entity certificate is also trusted, as the trust chain between them is continuous.

Certificate Authority (CA)

A digital certificate issuing authority is an authoritative body responsible for issuing and managing digital certificates, and as a trusted third party in e-commerce transactions, it bears the responsibility for verifying the legality of public keys in the public key system.

The CA center issues a digital certificate to each user who uses a public key, the function of the digital certificate is to prove that the user listed in the certificate legally owns the public key listed in the certificate. The CA is responsible for issuing, certifying, and managing issued certificates. It needs to formulate policies and specific steps to verify and identify user identities and sign user certificates to ensure the identity of the certificate holder and the ownership of the public key.

Whether a Digital Signature Needs a CA

It is not necessary. When there is not a third-party notary, a CA is not needed, and we can use a self-signed certificate. With ComPDFKit, you can manually set to trust self-signed certificates, which is very useful for trusted parties to sign and check files. However, since there is no digital certificate issuing authority for certification, self-signed digital identity cards cannot guarantee the validity of identity information, and they may not be accepted

in some use cases.

How to Confirm the Identity of the Digital Certificate Creator

Subject contains identity information about the certificate holder, commonly including fields such as C (Country), ST (Province), L (Locality), O (Organization), OU (Organizational Unit), CN (Common Name), and others. These details help identify who the certificate holder is. DN (Distinguished Name) represents the complete and hierarchical representation of the "Subject" field. It includes all the information from the "Subject" field and organizes it in a structured manner.

The X.509 standard specifies a specific string format for describing DN, for example:

```
CN=Alan, OU=RD Department, O=ComPDFKit, C=SG, Email=xxxxxx@example.com
```

3.10.3 Create Digital Certificates

PKCS12 (Public Key Cryptography Standard #12) format digital certificates usually contain a public key, a private key, and other information related to the certificate. PKCS12 is a standard format used to store security certificates, private keys, and other related information. This format is commonly used to export, backup, and share digital certificates and private keys which are used in secure communications and identity verification.

When creating a PKCS12 standard certificate, in addition to the data confirming your identity, a password is typically required to protect your certificate. Only those who possess the password can access the private key contained within and perform actions such as signing documents through the certificate.

This example shows how to create digital certificates:

```
// Generate certificate.
//
// Password: ComPDFKit
//
// info: /C=SG/O=ComPDFKit/D=R&D Department/CN=Alan/emailAddress=xxxx@example.com
//
// C=SG: This represents the country code "SG," which typically stands for Singapore.
// O=ComPDFKit: This is the Organization (O) field, indicating the name of the organization or entity, in this case, "ComPDFKit."
// D=R&D Department: This is the Department (D) field, indicating the specific department within the organization, in this case, "R&D
   Department."
// CN=Alan: This is the Common Name (CN) field, which usually represents the name of the individual or entity. In this case, it is "Alan."
// emailAddress=xxxx@example.com: Email is xxxx@example.com
//
// CPDFCertUsage.CPDFCertUsageAll: Used for both digital signing and data validation simultaneously
//
// is_2048 = true: Enhanced security encryption
//
String password = "ComPDFKit";
CPDFOwnerInfo ownerInfo = new CPDFOwnerInfo();
ownerInfo.setCommonName("Alan");
ownerInfo.setOrgnize("ComPDFKit");
ownerInfo.setEmail("xxxx@example.com");
ownerInfo.setCountry("SG");
ownerInfo.setOrgnizeUnit("R&D Department");
CPDFSignature.CertUsage flag = CPDFSignature.CertUsage.PDFDigSig;
File certFile = new File(rootDir + "/certificate/Certificate.pfx");
certFile.getParentFile().mkdirs();
boolean result = CPDFSignature.generatePKCS12Cert(ownerInfo, password, certFile.getAbsolutePath(), flag,true);
System.out.println("generatePKCS12Cert result: " + result);
```

3.10.4 Create Digital Signatures

Creating a digital signature involves two steps:

1. Create a Signature Field
2. Sign within the Signature Field

By following these two steps, you can either self-sign a document or invite others to sign within the signature field you've created.

Create a Signature Field

ComPDFKit offers support for customizing the styles of the signature form field and allows you to customize the appearance of your signature using drawn, image, and typed signatures.

This example shows how to create a signature field:

```
// Create a signature field.
//
// Page Index: 0
// Rect: CRect(28, 420, 150, 370)
// Border RGB:{ 0, 0, 0 }
// Widget Background RGB: { 150, 180, 210 }
//
CPDFDocument document = new CPDFDocument();
document.open(FileUtils.getAssetsTempFile("CommonFivePage.pdf"));
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF signatureRect = new RectF(28, 420, 150, 370);
CPDFSignatureWidget signatureWidget = (CPDFSignatureWidget)
    cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_SignatureFields,document);
signatureWidget.setFieldName("Signature1");
signatureWidget.setFillColor(0xFF96B4D2);
signatureWidget.setRect(signatureRect);
signatureWidget.updateAp();
```

Sign Within the Signature Field

To sign within the signature field, you need to do three things:

- Possess a certificate that conforms to the PKCS12 standard (in PFX or P12 format) and ensure that you know its password. You can create a compliant digital certificate using the built-in methods within the ComPDFKit SDK.
- Set the appearance of the digital signature.
- Write the data into the signature field.

This example shows how to sign within the signature field:

```
//     Sign in the signature field.
//
//     Text: Grantor Name
//     Content:
//     Name: get grantor name from certificate
//     Date: now(yyyy.mm.dd)
//     Reason: I am the owner of the document
//     DN: Subject
//     IsContentAlginLeft: false
//     IsDrawLogo: True
//     LogoBitmap: logo.png
//     text color RGB: { 0, 0, 0 }
//     Output file name: document.FileName + "_Signed.pdf"
//
// Make a digital signature.
String fileName = new File(document.getFilePath()).getName();
File file = new File(rootDir + "/digitalSignature/" + fileName);
file.getParentFile().mkdirs();
String password = "ComPDFKit";

String certPath = rootDir+"/certificate/Certificate.pfx";
CPDFX509 cpdfx509 = CPDFSignature.getX509ByPKCS12Cert(certPath, password);
String location = cpdfx509.getCertInfo().getSubject().getCountry();
String reason = "I am the owner of the document.";
BufferedImage image = ImageIO.read(new File(rootDir + "/TestFiles/ComPDFKit.png"));
Bitmap bitmap = new Bitmap();
int width = image.getWidth();
int height = image.getHeight();
```

```
int[] pixels = new int[width * height];
bitmap.setWidth(width);
bitmap.setHeight(height);
bitmap.setPixels(image.getRGB(0, 0, width, height, pixels, 0, width));

// Describe signature appearance information.
CPDFDigitalSigConfig sigConfig = new CPDFDigitalSigConfig();
sigConfig.setText(cpdfx509.getCertInfo().getSubject().getCommonName());
sigConfig.setTextColor(0xFF000000);
sigConfig.setContentColor(0xFF000000);
sigConfig.setLogo(bitmap);
String builder = "Name: " + cpdfx509.getCertInfo().getSubject().getCommonName() + "\n" +
    "Date: " + "2024.04.16 18:20:30" + "\n" +
    "Reason: " + reason + "\n" +
    "Location: " + location + "\n" +
    "DN: " + cpdfx509.getCertInfo().getSubject();
sigConfig.setContent(builder);
sigConfig.setContentAlginLeft(false);
sigConfig.setDrawLogo(true);
System.out.println("updateApWithDigitalSigConfig.");
boolean updateSignAp = signatureWidget.updateApWithDigitalSigConfig(sigConfig);
if (updateSignAp){
    boolean writeSignResult = document.writeSignature(
        signatureWidget,
        file.getAbsolutePath(),
        certPath,
        password,
        location,
        reason,
        CPDFDocument.PDFDocMdpP.PDFDocMdpPForbidAllModify
    );
    System.out.println("signWithinTheSignatureField result: " + writeSignResult);
}
```

3.10.5 Read Digital Signature Information

You can read various pieces of information from a document's digital signature, including the signature itself, the signer of the signature, and certain details of the signer's digital certificate.

For a comprehensive list of retrievable information, please refer to the API Reference.

This example shows how to read digital signature information:

```
CPDFDocument document = new CPDFDocument();
document.open(FileUtils.getAssetsTempFile("Signed.pdf"));
for (int i = 0; i < document.getSignatureCount(); i++) {
    CPDFSignature signature = document.getPdfSignature(i);
    // Check if the signer array exists and is not empty
    if (signature.getSignerArr() != null && signature.getSignerArr().length > 0) {
        CPDFSigner signer = signature.getSignerArr()[0];
        CPDFOwnerInfo subject = signer.getCert().getCertInfo().getSubject();
        System.out.println("Name: " + signature.getName());
        System.out.println("Location: " + signature.getLocation());
        System.out.println("Reason: " + signature.getReason());
        System.out.println("Date: " + signature.getDate());
        System.out.println("Subject: " + subject.toString());
    }
}
```

The Connection Between Digital Signatures, Signers, and Digital Certificates

A digital signature is generated by encrypting a document using the private key of the signer and then verifying the validity of the signature using the public key from the signer's certificate. The signature, signer, and digital certificate constitute a crucial part of digital signatures in a PDF document.

In most cases, one signature corresponds to one signer. However, in some situations, a digital signature can include multiple signers, each with their own certificate chain. This multi-signer mechanism can be very useful in

certain application scenarios because it allows multiple entities to digitally sign the same document, each using theircertificate and private key.

3.10.6 Verify Digital Certificates

When verifying digital certificates, the system automatically checks the trustworthiness of all certificates in the certificate chain and also verifies whether the certificates have expired. Only certificates that are both not expired and considered trustworthy in the entire certificate chain are considered trusted digital certificates.

This example shows how to verify digital certificates:

```
// Verify whether the specified certificate file is trusted.
//
// To verify the trustworthiness of a certificate, you need to validate that all certificates in the certificate chain are trustworthy.
String certFilePath = rootDir+"/certificate/Certificate.pfx";
String password = "ComPDFKit";
if (CPDFSignature.checkPKCS12Password(certFilePath, password)) {
    CPDFX509 x509 = CPDFSignature.getX509ByPKCS12Cert(certFilePath, password);
    boolean isTrusted = x509.checkCertificateIsTrusted(rootDir + "/trusted");
    if (isTrusted) {
        // Certificate is trusted.
        System.out.println("certificate is in trusted list");
    } else {
        // Certificate is trusted.
        System.out.println("not in");
    }
}
```

3.10.7 Verify Digital Signatures

Verifying a digital signature consists of signature validity and certificate trustworthiness. Signature validity indicates that the document has not been tampered with. Certificate trustworthiness confirms that the signer is trustworthy.

Generally, a signature is verified only when both the signature is valid and the certificate is trustworthy.

This example shows how to verify digital signatures:

```
CPDFDocument document = new CPDFDocument();
document.open(FileUtils.getAssetsTempFile("Signed.pdf"));
// Iterate through all digital signatures.
for (int i = 0; i < document.getSignatureCount(); i++) {
    CPDFSignature signature = document.getPdfSignature(i);
    // Check if the signer array exists and is not empty.
    if (signature.getSignerArr() != null && signature.getSignerArr().length > 0) {
        CPDFSigner signer = signature.getSignerArr()[0];

        // Verify the validity of the signature.
        boolean verifyValid = signature.verify(document);

        // Verify if the document has not been modified.
        boolean unmodified = signature.verifyDocument(document);

        // Determine if the signature is valid and the document is unmodified.
        boolean isSignVerified = verifyValid && unmodified;

        // Check if the certificate is trusted.
        boolean certChainTrusted = signer.getCert().verifyGetChain(rootDir+"/certificate/", signature);
        boolean certificateIsTrusted = signer.getCert().checkCertificateIsTrusted(rootDir + "/trusted");
        boolean certIsTrusted = certChainTrusted || certificateIsTrusted;

        // Check if the certificate has expired.
        boolean isExpired = signer.getCert().isExpired();
        if (isExpired) {
            System.out.println("the certificate is expired.");
        }else {
            System.out.println("the certificate not expired.");
```

```
        }

        // Take appropriate actions based on the verification results.
        if (isSignVerified && certIsTrusted) {
            // Signature is valid and the certificate is trusted.
            // Perform the corresponding actions.
            System.out.println("Signature is valid and the certificate is trusted.");
            System.out.println("Perform the corresponding actions.");
        } else if (isSignVerified && !certIsTrusted) {
            // Signature is valid but the certificate is not trusted.
            // Perform the corresponding actions.
            System.out.println("Signature is valid but the certificate is not trusted.");
            System.out.println("Perform the corresponding actions.");
        } else {
            // Signature is invalid.
            // Perform the corresponding actions.
            System.out.println("Signature is invalid.");
            System.out.println("Perform the corresponding actions.");
        }
    }
}
```

3.10.8 Trust Certificate

Trusting certificates involve two steps:

1. Specify the trust path (folder) for certificates. This path serves as the location where certificates are placed when they are trusted. Additionally, when checking the trustworthiness of certificates, the SDK will look for the corresponding certificates within this folder. Please ensure that this path is valid. If the path does not exist or is inaccessible, the ComPDFKit SDK will not automatically create the trust path folder.
2. Execute the method to trust certificates, and the certificates will be added to the trust path.

This example shows how to trust certificates:

```
// Add the certificate in the local storage as a trusted certificate.
String certFilePath = rootDir+"/certificate/Certificate.pfx";
String password = "ComPDFKit";
if (CPDFSignature.checkPKCS12Password(certFilePath, password)) {
    CPDFX509 x509 = CPDFSignature.getX509ByPKCS12Cert(certFilePath, password);
    // Add your certificate to the trust path.
    boolean result = x509.addToTrustedCertificates(rootDir + "/trusted","compdfkit.pem");
    if (result) {
        System.out.println("add to trusted certificates success");
    }else {
        System.out.println("add to trusted certificates fail");
    }
}
// Add digital signature certificate in pdf file as trusted certificate.
CPDFSignature signature = document.getPdfSignature(0);
CPDFSigner signer = signature.getSignerArr()[0];
CPDFX509 cpdfx509 = signer.getCert();

boolean result = cpdfx509.addToTrustedCertificates(rootDir + "/trusted","compdfkit.pem");
if (result) {
    System.out.println("add to trusted certificates success");
}else {
    System.out.println("add to trusted certificates fail");
}
```

3.10.9 Remove Digital Signatures

You can easily remove a digital signature, and when you do so, both the appearance and data associated with the signature will be deleted.

It's important to note that removing a signature does not remove the signature field.

This example shows how to remove digital signatures:

```
// Remove digital signature.
CPDFDocument document = new CPDFDocument();
document.open(FileUtils.getAssetsTempFile("Signed.pdf"));
for (int i = 0; i < document.getSignatureCount(); i++) {
    CPDFSignature signature = document.getPdfSignature(i);
    if (i == 0){
        // Remove first digital signature.
        boolean result = document.removeSignature(signature, document, true);
        if (result) {
            System.out.println("remove signature success");
        }else {
            System.out.println("remove signature fail");
        }
    }
}
document.save(rootDir + "/out/signatureTest.pdf", CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
```

3.10.10 Invisible Digital Signatures

An invisible digital signature does not display signature text, an image, a logo, or any other visible signature appearance on the page. The signature information is embedded in the PDF and can be inspected and verified through a PDF reader's signature panel.

The SDK creates a standard PDF signature form field (/FT /Sig) and its Widget annotation (/Subtype /Widget). The field value /V refers to a signature dictionary containing /Contents and /ByteRange. Setting the Widget rectangle to [0, 0, 0, 0] gives it zero width and height, following the PDF specification's field and annotation representation for invisible signatures. See section 1.2 of the PDF Association's Digital Signatures technical note for this representation.

Invisibility affects the page appearance. It does not change the signature's cryptographic integrity or the requirements for trusting its certificate.

Step 1: Prepare the Java application and signing materials

Initialize ComPDFKit as described in section 2.4 before running the following Java code. Prepare:

- A saved input PDF containing at least one page.
- A PKCS#12 identity (.p12 or .pfx) containing the private key, signer certificate, and any required intermediate certificates, together with its password.
- A directory of independently approved trusted certificates in PEM format for certificate-chain verification.
- An output path different from the input path.

The following blocks form one sequential workflow inside a Java method that declares throws IOException. Use these imports:

```
import com.compdfkit.core.android.graphics.RectF;
import com.compdfkit.core.annotation.form.CPDFSignatureWidget;
import com.compdfkit.core.annotation.form.CPDFWidget;
import com.compdfkit.core.document.CPDFDocument;
import com.compdfkit.core.signature.CPDFSignature;
import com.compdfkit.core.signature.CPDFSigner;
import com.compdfkit.core.signature.CPDFX509;
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
```

Configure the paths and password in the application:

```
File input = new File("input.pdf");
File output = new File("output/invisible-signed.pdf");
File certificateFile = new File("certs/signer.p12");
String certificatePassword = "YOUR_PKCS12_PASSWORD";
File trustDirectory = new File("certs/trusted");
String trustedPath = trustDirectory.getAbsolutePath() + File.separator;

if (!input.isFile() || !input.canRead()) {
    throw new IllegalStateException("Cannot read the input PDF");
}
if (input.getCanonicalFile().equals(output.getCanonicalFile())) {
    throw new IllegalArgumentException("Use different input and output paths");
}
if (!trustDirectory.isDirectory()) {
    throw new IllegalStateException("Prepare the trusted certificate directory");
}
Files.createDirectories(output.getAbsoluteFile().getParentFile().toPath());
```

Step 2: Import the certificate and check its password and validity

Import means reading the signing identity from the supplied PKCS#12 file. It does not install the identity into the operating system or make its certificate trusted.

```
if (!certificateFile.isFile() || !certificateFile.canRead()) {
    throw new IllegalStateException("Cannot read the PKCS#12 file");
}
if (!CPDFSignature.checkPKCS12Password(
        certificateFile.getAbsolutePath(), certificatePassword)) {
    throw new IllegalStateException("Cannot unlock the PKCS#12 file");
}
CPDFX509 signingCertificate = CPDFSignature.getX509ByPKCS12Cert(
        certificateFile.getAbsolutePath(), certificatePassword);
if (signingCertificate == null || !signingCertificate.isValid()) {
    throw new IllegalStateException("Cannot import the signing certificate");
}
if (signingCertificate.isExpired()) {
    throw new IllegalStateException("Certificate is outside its validity period");
}
System.out.println("Signing certificate: "
        + signingCertificate.getCertInfo().getSubject().getCommonName());
```

getX509ByPKCS12Cert() reads the certificate for inspection. writeSignature() below loads the private key and signing identity from the same PKCS#12 file.

Step 3: Create the invisible signature field and apply the signature

Set the Widget rectangle to zero and do not configure a visible text, image, or logo appearance. The output below forbids subsequent document modifications through its DocMDP permission setting.

```
CPDFDocument document = new CPDFDocument();
try {
    document.open(input.getAbsolutePath());
    if (!document.isValid() || document.getPageCount() == 0) {
        throw new IllegalStateException("Cannot open a PDF page for signing");
    }
    CPDFSignatureWidget widget = (CPDFSignatureWidget) document.pageAtIndex(0)
            .addFormWidget(CPDFWidget.WidgetType.Widget_SignatureFields, document);
    if (widget == null || !widget.isValid()) {
        throw new IllegalStateException("Cannot create the signature field");
    }
    widget.setFieldName("InvisibleSignature");
    widget.setRect(new RectF(0, 0, 0, 0));

    boolean signed = document.writeSignature(
            widget,
            output.getAbsolutePath(),
            certificateFile.getAbsolutePath(),
            certificatePassword,
            "Signing location",
            "Approval",
            CPDFDocument.PDFDocMdpP.PDFDocMdpPForbidAllModify);
    if (!signed) {
        throw new IllegalStateException("Invisible signature creation failed");
    }
} finally {
    document.close();
}
```

Step 4: Reopen the output and verify the signature

Verify the exact saved output. Check cryptographic integrity, document modification permissions, certificate validity, and certificate-chain trust separately. The trust check uses the certificate embedded in the signed PDF and the configured trust directory.

```
CPDFDocument signedDocument = new CPDFDocument();
try {
    signedDocument.open(output.getAbsolutePath());
    if (!signedDocument.isValid() || signedDocument.getSignatureCount() != 1) {
        throw new IllegalStateException("Expected one signature in the output PDF");
    }
    CPDFSignature signature = signedDocument.getPdfSignature(0);
    if (signature == null || !signature.isValid()) {
        throw new IllegalStateException("Cannot read the signature");
    }
    CPDFSigner[] signers = signature.getSignerArr();
    if (signers == null || signers.length == 0) {
        throw new IllegalStateException("The signature has no signer");
    }

    boolean integrity = signature.verify(signedDocument);
    boolean permissionsValid = signature.verifyDocument(signedDocument);
    boolean certificatesTrusted = true;
    boolean certificatesWithinValidity = true;
    for (int i = 0; i < signers.length; i++) {
        if (signers[i] == null || !signers[i].isValid()) {
            throw new IllegalStateException("Cannot read the signer");
        }
        CPDFX509 certificate = signers[i].getCert();
        if (certificate == null || !certificate.isValid()) {
            throw new IllegalStateException("Cannot read the signer certificate");
        }
        boolean withinValidity = !certificate.isExpired();
        boolean chainTrusted = certificate.verifyGetChain(trustedPath, signature);
        certificatesWithinValidity &= withinValidity;
        certificatesTrusted &= chainTrusted;
        System.out.println("Signer " + i + " certificate within validity: "
                + withinValidity);
        System.out.println("Signer " + i + " certificate chain trusted: "
                + chainTrusted);
    }
    boolean verified = integrity && permissionsValid
            && certificatesWithinValidity && certificatesTrusted;
    System.out.println("Signature integrity: " + integrity);
    System.out.println("Document permissions valid: " + permissionsValid);
    System.out.println("Basic verification passed: " + verified);
    if (!verified) {
        throw new IllegalStateException("Invisible signature verification failed");
    }
} finally {
    signedDocument.close();
}
```

This is basic signature verification. It does not check revocation, establish a trusted signing time, or verify an LT/LTA profile. Use section 3.10.11 for LTV validation.

Run the sample and inspect the result

samples/SignatureTest/SignatureTest.java implements the complete workflow in signInvisibleDigitalSignature() and verifyInvisibleDigitalSignature(). From samples/SignatureTest, run ./RunTest.sh. It creates a test identity, imports it, applies the invisible signature, reopens the output, and prints each verification result. The output is samples/out/SignatureTest/digitalSignature/Invisible_Signed.pdf.

The sample explicitly trusts its generated self-signed test identity in samples/SignatureTest/trusted/invisible-demo.pem. For an existing identity, edit INVISIBLE_CERTIFICATE_PATH, INVISIBLE_CERTIFICATE_PASSWORD, and INVISIBLE_TRUST_DIRECTORY, then set TRUST_INVISIBLE_DEMO_CERTIFICATE to false and supply independently approved trust anchors. These are fixed code settings, not additional command-line arguments.

Open the output's signature panel in Acrobat to inspect the signature information. The PDF page displays no visible signature appearance. Trust configured for this Java sample does not change Acrobat's trust settings, so Acrobat

may still report an unknown signer until its own trust configuration is prepared.

3.10.11 Long-Term Validation (LTV) and PAdES

Long-term validation preserves certificate and revocation evidence needed to validate a signature later. The Java SDK provides PAdES creation, LTV prolongation, material assessment, and detailed validation reports.

Profile	Content	Output
B	Baseline signature with a signing-certificate reference	Signed PDF
T	B plus an embedded RFC 3161 signature timestamp	Signed PDF
LT	T plus certificate and revocation evidence in DSS/VRI	LTV PDF
LTA	LT plus an archival document timestamp protecting the evidence	LTA PDF

Initialize the SDK before calling these APIs. Signing, evidence collection, and TSA requests are synchronous, so run them on a worker thread. Save document edits first and keep the input file unchanged during the operation.

Each requested output must use a distinct new path in an existing directory. These high-level entry points do not accept a PDF-open password. The PKCS#12 password unlocks the signing certificate container, not the PDF.

Create a PAdES-LT Signature

The following example creates an invisible PAdES-LT signature. Replace the certificate, trust bundle, TSA endpoint, and CRL URL with resources appropriate to your signing identity.

```
Files.createDirectories(Paths.get("output"));
CPDFSignatureValidationOptions validation = new CPDFSignatureValidationOptions()
        .setTrustedCertPath("certs/trusted-roots.pem")
        .setCheckRevocation(true);

CPDFPadesOptions options = new CPDFPadesOptions()
        .setProfile(CPDFPadesOptions.Profile.LT)
        .setCertificate("certs/signer.p12", "YOUR_PKCS12_PASSWORD")
        .setFieldName("PadesSignature")
        .setVisible(false)
        .setTsaUrl("https://tsa.example.com/timestamp")
        .setTsaTransport(20, 0, "")
        .setTrustedCertPath("certs/trusted-roots.pem")
        .setRevocationUrls(null, new String[] {"https://ca.example.com/issuer.crl"})
        .setValidationOptions(validation);

CPDFPadesResult result = CPDFPadesSigner.sign(
        "input.pdf",
        "output/pades-signed.pdf",
        "output/pades-lt.pdf",
        null,
        options);
System.out.println("Process success: " + result.isSuccess());
System.out.println("Target profile satisfied: " + result.isTargetProfileSatisfied());
System.out.println("Final completed output: " + result.getFinalPath());
System.out.println(Arrays.toString(result.getErrors()));
System.out.println(result.getValidationResult());
```

For LTA creation, select Profile.LTA and supply a new LTA output path as the fourth argument to sign(). Configure an archival TSA through setLtaTsaUrl() when it differs from the signature TSA.

The trust bundle must contain the trust anchors required for the signer and TSA chains. Certificates embedded in a PDF or CMS container do not become trusted merely because they are embedded.

Prolong LTV for an Existing Signature

Use prolongLtv() to collect or update evidence for an existing signed PDF. It does not create another approval signature or require a new PKCS#12 signer. The existing signature must be suitable for the requested profile; adding evidence does not rebuild its signed attributes.

```
Files.createDirectories(Paths.get("output"));
CPDFPadesOptions options = new CPDFPadesOptions()
        .setTrustedCertPath("certs/trusted-roots.pem")
        .setRevocationUrls(null, new String[] {"https://ca.example.com/issuer.crl"})
        .setTsaUrl("https://tsa.example.com/timestamp")
        .setTsaTransport(20, 0, "")
        .setAppendDocumentTimestamp(true);

CPDFPadesResult result = CPDFPadesSigner.prolongLtv(
        "existing-pades-t.pdf",
        "output/prolonged-lt.pdf",
        "output/prolonged-lta.pdf",
        options);
System.out.println("Process success: " + result.isSuccess());
System.out.println("Target profile satisfied: " + result.isTargetProfileSatisfied());
System.out.println(result.getValidationResult());
```

Set appendDocumentTimestamp to false and omit the LTA path when only extending LT evidence. If a later stage fails, an earlier successfully completed stage may remain available. Inspect the error list and actual completed paths rather than treating every configured output path as a successful output.

Assess Embedded Materials

CPDFLtvService.assessReadiness() reports material availability. It does not establish certificate trust, revocation validity, or a verified PAdES profile.

```
CPDFDocument document = new CPDFDocument();
try {
    document.open("output/pades-lt.pdf");
    CPDFLtvReadinessResult result = CPDFLtvService.assessReadiness(document);
    if (result != null && result.isSuccess()) {
        System.out.println("LTV materials ready: "
                + result.getReadinessInfo().isLtvReady());
    }
} finally {
    document.close();
}
```

Read Detailed Validation Reports

Use CPDFSignatureValidator to inspect the exact saved PDF bytes. Validation reads embedded evidence; it does not collect new OCSP/CRL material. Enable revocation checks for an LTV verdict and configure native trust anchors explicitly.

```
CPDFSignatureValidationOptions options = new CPDFSignatureValidationOptions()
        .setTrustedCertPath("certs/trusted-roots.pem")
        .setCheckRevocation(true);
CPDFSignatureValidationResult result = CPDFSignatureValidator.validate(
        "output/pades-lt.pdf", options);
System.out.println("Overall status: " + result.getOverallStatus());
for (CPDFSignatureValidationReport report : result.getReports()) {
    System.out.println("Field: " + report.getFieldName());
    System.out.println("SubFilter: " + report.getSubFilter());
    System.out.println("Status: " + report.getOverallStatus());
    System.out.println("LTV enabled: " + report.isLtvEnabled());
    for (CPDFValidationReportItem item : report.getItems()) {
        System.out.println(item);
    }
}
```

Process success, material readiness, target-profile satisfaction, and complete validation are separate results. isTargetProfileSatisfied() is a profile helper; for LTA, inspect the document timestamp reports as well. Strict acceptance should require the target evidence and valid reports for all signatures and timestamps relevant to the application.

The statuses INVALID, INDETERMINATE, and NOT_CHECKED have different meanings. A signature can have valid cryptographic integrity while other checks remain indeterminate, including missing LTV evidence. The SDK does not convert that state to VALID.

See SignatureLtvKmsTest for a complete LT creation and prolongation example.

3.10.12 AWS KMS Signing

CPDFAwsKmsSigner signs a saved PDF through an AWS KMS key. The signing private key stays with KMS. The application supplies the KMS configuration and a signer certificate whose public key matches that key.

The current direct binding supports an RSA KMS key of at least 2048 bits and RSASSA_PKCS1_V1_5_SHA_256. Other signing algorithms are rejected. Its endpoint must be a root HTTPS URL without an explicit port, extra path, query, or fragment.

The direct binding requires explicit access-key credentials. It does not automatically use the AWS default credential provider chain. Temporary credentials with a nonempty session token are currently rejected with sdk.kms.session_token_unsupported.

Load credentials from your application's configuration. Do not place real secret keys in source code or logs.

```
Files.createDirectories(Paths.get("output"));
CPDFAwsKmsConfig kms = new CPDFAwsKmsConfig()
        .setEndpoint("https://kms.us-east-1.amazonaws.com")
        .setRegion("us-east-1")
        .setKeyId("YOUR_KMS_KEY_ID_OR_ARN")
        .setCredentials(
                System.getenv("AWS_ACCESS_KEY_ID"),
                System.getenv("AWS_SECRET_ACCESS_KEY"),
                "")
        .setSigningAlgorithm("RSASSA_PKCS1_V1_5_SHA_256")
        .setCertificate(
                Files.readAllBytes(Paths.get("certs/kms-signer.der")),
                new byte[][] {Files.readAllBytes(Paths.get("certs/kms-issuer.der"))});

CPDFSignatureOptions options = new CPDFSignatureOptions()
        .setFieldName("KmsSignature")
        .setVisible(false)
        .setValidationOptions(new CPDFSignatureValidationOptions()
                .setTrustedCertPath("certs/trusted-roots.pem"));
CPDFSignatureCreateResult result = CPDFAwsKmsSigner.sign(
        "input.pdf", "output/kms-signed.pdf", options, kms);
System.out.println("Creation success: " + result.isSuccess());
System.out.println(Arrays.toString(result.getErrors()));
System.out.println(result.getValidationResult());
```

Use DER-encoded signer and chain certificates. The output must be new, different from the input, and have an existing parent directory. Creation success is separate from signature validation and trust.

Fetch a Public Key or Generate a CSR

getPublicKey() returns DER-encoded X.509 SubjectPublicKeyInfo. generateCsr() returns a signed PKCS#10 request in DER; submit it to a CA to obtain a certificate. Neither method exports the private key or issues a CA certificate.

Using the kms configuration from the previous example:

```
CPDFKmsDataResult publicKey = CPDFAwsKmsSigner.getPublicKey(kms);
if (publicKey.isSuccess()) {
    Files.write(Paths.get("output/kms-public-key.der"), publicKey.getData());
}

kms.setCommonName("Example PDF Signing Key");
CPDFKmsDataResult csr = CPDFAwsKmsSigner.generateCsr(kms);
if (csr.isSuccess()) {
    Files.write(Paths.get("output/kms-signer.csr.der"), csr.getData());
}
```

The current CPDFAwsKmsSigner.sign() creates an ordinary detached PKCS#7 signature. It does not create PAdES attributes, embedded signature timestamps, or LTV evidence. signPades() currently returns sdk.kms.pades_unsupported without contacting KMS or creating a PDF.

Adding a document timestamp or calling LTV prolongation is not, by itself, a guarantee that an ordinary KMS signature satisfies LT/LTA. Inspect the profile reports. When an external signing service manages KMS and supplies a complete CMS container, use the external container or two-phase APIs; the Java client does not need that service's internal KMS credentials.

3.10.13 Two-Phase External Signing

Two-phase signing separates PDF preparation from the external signature operation. It supports delayed responses, another JVM, or another machine without keeping the original document or a native pointer alive.

1. Prepare: create a new signature field, reserve /Contents, set /ByteRange, and return the digest and portable Session.
2. External signing: obtain a signature for this preparation from the external service.
3. Fill: write the returned CMS container or raw signature into the unchanged prepared PDF.
4. Validate: inspect the complete verification report.

Hash mode	What the external service signs	Fill method
DOCUMENT_DIGEST	Digest of the PDF bytes covered by /ByteRange	Complete CMS or raw signature value
SIGNED_ATTRIBUTES	Digest of frozen CMS signed attributes	Raw signature value

Supported digest algorithms are SHA-256, SHA-384, and SHA-512. The raw RSA contract is PKCS#1 v1.5; the current interface does not represent RSA-PSS parameters.

Prepare a Signature

Signed-attributes mode requires the signer certificate during preparation because the attributes bind that certificate. The default field is invisible. The default reservation is 65536 binary bytes for the complete CMS container.

```
Files.createDirectories(Paths.get("output"));
CPDFExternalSignPrepareOptions options = new CPDFExternalSignPrepareOptions()
        .setFieldName("ExternalSignature")
        .setHashMode(CPDFExternalSignPrepareOptions.HashMode.SIGNED_ATTRIBUTES)
        .setDigestAlgorithm(CPDFSignatureDigest.SHA256)
        .setEstimatedContentsSize(65536)
        .setCertificate(
                Files.readAllBytes(Paths.get("certs/signer.der")),
                new byte[][] {Files.readAllBytes(Paths.get("certs/issuer.der"))});

CPDFExternalSignPrepareResult prepared = CPDFExternalSignSession.prepare(
        "input.pdf", "output/prepared.pdf", options);
if (!prepared.isSuccess()) {
    throw new IllegalStateException(prepared.getErrorCode() + ": " + prepared.getErrorMessage());
}
CPDFExternalSignSession.saveSession("output/signing-session.txt", prepared.getSession());
Files.write(Paths.get("output/hash-to-sign.bin"), prepared.getHashToSign());
```

```
System.out.println("Digest algorithm: " + prepared.getDigestAlgorithm());
System.out.println("Hash to sign: "
        + Base64.getEncoder().encodeToString(prepared.getHashToSign()));
```

The hash is already digested. Do not hash it again as an ordinary message and do not sign its Base64 text. Keep the prepared PDF, complete Session, hash, and digest algorithm associated with the same transaction. Repeating Prepare may change the bytes, signing time, and hash.

saveSession() creates a new UTF-8 file and does not overwrite an existing one. The Session is the Core format, not a Java object serialization. Preserve it without trimming or rewriting; the application is responsible for saving the transaction successfully.

Fill an External Raw Signature

This example can run in another process after the service has returned external-signature.bin for the preceding preparation:

```
String session = CPDFExternalSignSession.loadSession("output/signing-session.txt");
byte[] signatureValue = Files.readAllBytes(Paths.get("external-signature.bin"));
CPDFExternalSignFillOptions options = new CPDFExternalSignFillOptions()
        .setValidationOptions(new CPDFSignatureValidationOptions()
                .setTrustedCertPath("certs/trusted-roots.pem"));
CPDFExternalSignFillResult result = CPDFExternalSignSession.fillWithSignatureValue(
        session,
        "output/prepared.pdf",
        "output/external-signed.pdf",
        signatureValue,
        options);
System.out.println("Fill success: " + result.isSuccess());
System.out.println(result.getErrorCode() + ": " + result.getErrorMessage());
System.out.println(result.getValidationResult());
```

For SIGNED_ATTRIBUTES, Core uses the certificate and attributes frozen in the Session. For DOCUMENT_DIGEST, supply the certificate at Prepare or through CPDFExternalSignFillOptions.setCertificate() at Fill.

The prepared file can move after preparation, provided its exact bytes and Session are preserved. It is kept after a failed Fill so that the application can retry with the correct payload.

Fill a Complete CMS Container

Prepare a separate DOCUMENT_DIGEST transaction when the external service returns a complete detached CMS/PKCS#7 container. A container with signed attributes must carry the correct messageDigest for the prepared PDF's /ByteRange.

```
Files.createDirectories(Paths.get("output"));
CPDFExternalSignPrepareResult prepared = CPDFExternalSignSession.prepare(
        "input.pdf",
        "output/container-prepared.pdf",
        new CPDFExternalSignPrepareOptions()
                .setHashMode(CPDFExternalSignPrepareOptions.HashMode.DOCUMENT_DIGEST));
if (!prepared.isSuccess()) {
    throw new IllegalStateException(prepared.getErrorCode());
}
CPDFExternalSignSession.saveSession("output/container-session.txt", prepared.getSession());

// Obtain this CMS from the service for the preparation above.
byte[] containerDer = Files.readAllBytes(Paths.get("external-container.der"));
CPDFExternalSignFillResult result = CPDFExternalSignSession.fillWithContainer(
        CPDFExternalSignSession.loadSession("output/container-session.txt"),
        "output/container-prepared.pdf",
        "output/container-signed.pdf",
        containerDer,
        new CPDFExternalSignFillOptions()
                .setVerifyMessageDigest(true)
                .setValidationOptions(new CPDFSignatureValidationOptions()
```

```
                .setTrustedCertPath("certs/trusted-roots.pem")));
System.out.println(result.getValidationResult());
```

The payload must be one complete detached SignedData DER value with signer information. Attached CMS, malformed containers, and extra trailing data are rejected. The reservation must fit the entire encoded container, including certificates and timestamps. A larger reservation requires a new Prepare operation and a new external signature.

Memory-Based Signing

Prepare and both Fill methods also have byte[] overloads. Retrieve prepared bytes with getPreparedPdf() and completed bytes with getOutputPdf(). File preparation and memory Fill, or memory preparation and file Fill, can use the same Core Session.

```
byte[] inputPdf = Files.readAllBytes(Paths.get("input.pdf"));
CPDFExternalSignPrepareResult prepared = CPDFExternalSignSession.prepare(
        inputPdf,
        new CPDFExternalSignPrepareOptions()
                .setHashMode(CPDFExternalSignPrepareOptions.HashMode.SIGNED_ATTRIBUTES)
                .setCertificate(
                        Files.readAllBytes(Paths.get("certs/signer.der")),
                        new byte[][] {Files.readAllBytes(Paths.get("certs/issuer.der"))}));
if (!prepared.isSuccess()) {
    throw new IllegalStateException(prepared.getErrorCode());
}

// Obtain the service response for prepared.getHashToSign().
byte[] signatureValue = Files.readAllBytes(Paths.get("external-memory-signature.bin"));
CPDFExternalSignFillResult result = CPDFExternalSignSession.fillWithSignatureValue(
        prepared.getSession(), prepared.getPreparedPdf(), signatureValue,
        new CPDFExternalSignFillOptions()
                .setValidationOptions(new CPDFSignatureValidationOptions()
                        .setTrustedCertPath("certs/trusted-roots.pem")));
if (result.isSuccess()) {
    Files.createDirectories(Paths.get("output"));
    Files.write(Paths.get("output/memory-signed.pdf"), result.getOutputPdf());
}
System.out.println(result.getValidationResult());
```

Memory Prepare and Fill do not write PDF files. Optional validation uses a seekable source containing the original bytes: an anonymous memory file on Linux when available, otherwise a private temporary byte-for-byte copy removed after validation. The PDF is not reserialized.

Fill success only means that a PDF was produced and, for file APIs, published. It does not prove that a returned signature is correct or trusted. Verification is enabled by default; inspect getValidationResult(). Disabled verification reports NOT_CHECKED.

Two-phase signatures use adbe.pkcs7.detached. They do not automatically collect LTV material or inject an embedded timestamp. An external service may supply a timestamped complete container, or the application may append an independent document timestamp after Fill.

See SignatureTwoPhaseTest for fixed-configuration Prepare/Fill operation and recovery.

3.10.14 Signature and Document Timestamps

The Java SDK exposes three distinct timestamp operations:

Operation	API	Meaning
Embedded signature timestamp	CPDFDocument.writeSignature(..., tsaUrl) or PAdES T/LT/LTA creation	A token inside the signature container binds the signature value
Independent document timestamp	CPDFTimestampService.addDocumentTimestamp()	A new ETSI.RFC3161 field protects the PDF revision

Operation	API	Meaning
Data or digest timestamp token	timestampData() / timestampDigest()	Returns a DER RFC 3161 token without modifying a PDF

A local signing-time attribute is not a trusted TSA timestamp.

Append a Document Timestamp

A document timestamp uses the TSA's identity and does not require the application's signing private key. It can be appended to the result of two-phase signing.

```
Files.createDirectories(Paths.get("output"));
CPDFTimestampOptions tsa = new CPDFTimestampOptions()
        .setTsaUrl("https://tsa.example.com/timestamp")
        .setTransport(20, 0, "");
CPDFSignatureOptions field = new CPDFSignatureOptions()
        .setFieldName("DocumentTime")
        .setValidationOptions(new CPDFSignatureValidationOptions()
                .setTrustedCertPath("certs/trusted-roots.pem"));
CPDFSignatureCreateResult result = CPDFTimestampService.addDocumentTimestamp(
        "output/external-signed.pdf",
        "output/timestamped.pdf",
        tsa,
        field);
System.out.println("Creation success: " + result.isSuccess());
System.out.println(result.getValidationResult());
```

The output must be new and different from the input. The current document-timestamp signer uses SHA-256; choosing another digest is explicitly rejected. This operation does not collect DSS/VRI evidence and does not, by itself, establish PAdES-LTA. A TSA failure does not publish a final PDF.

Request a Data or Digest Token

Data and digest requests support SHA-256, SHA-384, and SHA-512. Transport options include timeout, retries, proxy URL, and maximum response size. Request options include policy OID, nonce, and whether certificates should be included.

```
byte[] data = Files.readAllBytes(Paths.get("input.pdf"));
CPDFTimestampOptions options = new CPDFTimestampOptions()
        .setTsaUrl("https://tsa.example.com/timestamp")
        .setDigestAlgorithm(CPDFSignatureDigest.SHA256)
        .setTransport(20, 0, "");
CPDFTimestampResult result = CPDFTimestampService.timestampData(data, options);
if (result.isSuccess()) {
    Files.createDirectories(Paths.get("output"));
    Files.write(Paths.get("output/timestamp-token.der"), result.getTokenDer());
    System.out.println("TSA generation time: " + result.getGenerationTime());
} else {
    System.out.println(result.getErrorCode() + ": " + result.getErrorMessage());
}
```

If the application already has an imprint, call timestampDigest() with the raw digest bytes and the matching algorithm. That method does not hash the digest again.

A successful TSA response is separate from a certificate-trust verdict. For PDF timestamps, inspect the timestamp imprint, cryptographic signature, and TSA trust-chain items in the full validation report. HTTPS and certificate trust use native OpenSSL configuration rather than the JVM trust store.

See SignatureTimestampTest for embedded timestamps, document timestamps, and both token-request methods.

3.11 Text Search

3.11.1 Overview

ComPDFKit provides powerful PDF text search functionality, supporting case-sensitive/insensitive search, whole word matching, and sequential page searching to help users quickly locate specific content in documents.

Benefits of ComPDFKit Text Search

- Flexible Search Options: Support case-sensitive/insensitive and whole word matching search modes.
- Sequential Search: Search pages in order, suitable for large document searching.
- Accurate Positioning: Return the exact position information of search results for highlighting or navigation.

3.11.2 Search Text

Use CPDFTextSearcher class to perform text search. The search steps are as follows:

1. Create a CPDFTextSearcher object.
2. Iterate through each page and get the CPDFTextPage object.
3. Call findStart to begin searching.
4. Use findNext to iterate through search results.
5. Call findClose to end the search.

Here is an example of text search:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");

CPDFPage page = document.pageAtIndex(0);
CPDFTextPage textPage = page.getTextPage();

CPDFTextSearcher searcher = new CPDFTextSearcher();
boolean started = searcher.findStart(textPage, "PDF", 0,
        CPDFTextSearcher.PDFSearchOptions.PDFSearchCaseSensitive);

int count = 0;
CPDFTextRange range;
while ((range = searcher.findNext()) != null) {
    count++;
    System.out.println("Match " + count + ": location=" + range.location
            + ", length=" + range.length);
    System.out.println("  Text: " + textPage.getText(range));
}
searcher.findClose();
searcher.close();
textPage.close();
document.close();
}
```

3.12 Document Compression

3.12.1 Overview

ComPDFKit provides powerful PDF document compression and optimization functionality. Through the CPDFOptimizeOption class, you can finely control compression behavior to effectively reduce file size for easier storage and transmission.

Benefits of ComPDFKit Document Compression

• Fine-grained Image Compression: Support multiple compression algorithms (JPEG, JPEG2000, JBIG2, CCITT3/4), adjustable image quality, resolution, and block size.

• Rich Optimization Flags: Provides 30+ optimization flags, supporting removal of unused resources, cleaning metadata, merging layers, and other operations.

• Progress Callback Support: Get real-time compression progress through delegate functions, with support for cancellation.

• Web Optimization: Support PDF linearization for fast web viewing.

Feature Links

• Optimization Options Configuration

• Image Compression Settings

• Execute Document Compression

3.12.2 Optimization Options Configuration

Get Optimization Options

Use the CPDFDocument.getOptimizeOptions() method to get the document's optimization options object:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");
CPDFOptimizeOption optimizeOption = document.getOptimizeOptions();
```

Set Optimization Flags

Use the setOptimizeFlags() method to set optimization flags. Multiple flags can be combined using bitwise OR (|):

```
// Set a single flag
optimizeOption.setOptimizeFlags(CPDFOptimizeOptions.OPTIMIZE_FLAG_RMANNOT);

// Combine multiple flags
int combinedFlags =
    CPDFOptimizeOptions.OPTIMIZE_FLAG_RMNOTUSE |
    CPDFOptimizeOptions.OPTIMIZE_FLAG_RMEPTOBJ |
    CPDFOptimizeOptions.OPTIMIZE_FLAG_RMEMBFONT |
    CPDFOptimizeOptions.OPTIMIZE_FLAG_RMINVALINK;
optimizeOption.SetFlag(combinedFlags);

// Get current flags
int currentFlags = optimizeOption.getOptimizeFlags();
```

CPDF_OPTIMIZE_FLAG Optimization Flag Description

Flag	Value	Description
OPTIMIZE_FLAG_RMNOTUSE	1	Remove unused resources and elements
OPTIMIZE_FLAG_RMEPTOBJ	1<<1	Remove empty objects
OPTIMIZE_FLAG_RMSPEATTR	1<<2	Remove special attributes
OPTIMIZE_FLAG_RMEMBFONT	1<<3	Optimize embedded font memory usage
OPTIMIZE_FLAG_RMINVALINK	1<<4	Remove invalid links
OPTIMIZE_FLAG_RMINVABK	1<<5	Remove invalid bookmarks
OPTIMIZE_FLAG_BCOMPRESSIMAGE	1<<6	Enable image compression
OPTIMIZE_FLAG_RMBK	1<<7	Remove bookmarks
OPTIMIZE_FLAG_RMANNOT	1<<8	Remove annotations
OPTIMIZE_FLAG_RMFORM	1<<9	Remove forms
OPTIMIZE_FLAG_RMMULMEDIA	1<<10	Remove multimedia content
OPTIMIZE_FLAG_RMDOCINFO	1<<11	Remove document info (author, title, etc.)
OPTIMIZE_FLAG_RMMEDTADATA	1<<12	Remove metadata
OPTIMIZE_FLAG_RMOBJDATA	1<<13	Remove object data
OPTIMIZE_FLAG_RMFILEATTACHMENT	1<<14	Remove file attachments
OPTIMIZE_FLAG_RMEXTERNCROSSREF	1<<15	Remove external cross-references
OPTIMIZE_FLAG_RMOTHERAPPDATA	1<<16	Remove other application data
OPTIMIZE_FLAG_RMHIDERLAYER	1<<17	Remove hidden layers
OPTIMIZE_FLAG_MERGEVISIBLELAYER	1<<18	Merge visible layers
OPTIMIZE_FLAG_USEFLATE	1<<19	Use Flate (ZIP) compression
OPTIMIZE_FLAG_FLAT_REPLACELZW	1<<20	Replace LZW compression with Flate
OPTIMIZE_FLAG_RMUNUSEDTARGET	1<<21	Remove unused target objects
OPTIMIZE_FLAG_OPTIMIZEPAGECONTENT	1<<22	Optimize page content

Flag	Value	Description
OPTIMIZE_FLAG_OPTIMIZEPDFFASTWEBVIEW	1<<23	Enable fast web view optimization (linearization)
OPTIMIZE_FLAG_RMFORMCOMMITIMPORTRESETACTION	1<<24	Remove form submit/import/reset actions
OPTIMIZE_FLAG_RMJSACTION	1<<25	Remove JavaScript actions
OPTIMIZE_FLAG_RMPAGETHUMBNAIL	1<<26	Remove page thumbnails
OPTIMIZE_FLAG_RMREPLACEIMAGE	1<<27	Remove and replace images
OPTIMIZE_FLAG_RMLABEL	1<<28	Remove labels
OPTIMIZE_FLAG_RMPRINTSETTINGS	1<<29	Remove print settings
OPTIMIZE_FLAG_RMSEARCHINDEX	1<<30	Remove search index

3.12.3 Image Compression Settings

CPDFOptimizeImageOption Parameter Description

The CPDFOptimizeImageOption class is used to configure detailed image compression parameters:

Property	Type	Default	Description
uperPpi	int	225	Upper PPI (pixels per inch), images above this value will be downsampled
targetPpi	int	150	Target PPI, the target resolution after downsampling
compressAlgorithm	CPDFCompressAlgorithm	JPEG2000	Compression algorithm
quality	int	40	Image quality (1-100)
blockSize	int	256	Data block size (bytes)

CPDF_COMP_ALG Compression Algorithm Description

Algorithm	Value	Description	Use Case
COMP_ALG_JPEG2000	1	JPEG 2000 compression	High-quality color images, supports lossless and lossy compression
COMP_ALG_JPEG	2	Standard JPEG compression	Photos and color images, best compatibility
COMP_ALG_JBIG2	3	JBIG2 compression	Scanned documents and black-and-white images
COMP_ALG_CCITT4	4	CCITT Group 4 compression	Monochrome images, fax documents
COMP_ALG_CCITT3	5	CCITT Group 3 compression	Fax images, monochrome bitmaps

Configure Image Compression Options

```
// Create image optimization options
CPDFImageOptimizeOptions imageOption = new CPDFImageOptimizeOptions();
imageOption.upperPpi = 300;                         // Images above 300 PPI will be downsampled
imageOption.targetPpi = 150;                        // Downsample target is 150 PPI
imageOption.compressAlgorithm = CPDFCompressAlgorithm.JPEG2000;      // Use JPEG compression
imageOption.quality = 60;                           // Image quality 60%
imageOption.blockSize = 256;                        // Block size 256 bytes

// Apply image options
optimizeOption.setColorImageOptions(imageOption);
```

Quick Set Image Quality

If you only need to adjust image quality, you can use the shortcut method:

```
// Set image quality (1-100)
optimizeOption.setImageQuality(75.0f);

// Get current image quality
float quality = optimizeOption.getImageQuality();
```

3.12.4 Execute Document Compression

Basic Compression Example

Use the WriteOptimizeToFilePath() method to execute compression and save the document:

```
CPDFDocument document = new CPDFDocument();
document.open("test.pdf");
CPDFOptimizeOption optimizeOption = document.GetOptimizeOption();

// Configure optimization options
int flags =
    CPDFOptimizeOption.OPTIMIZE_FLAG_RMNOTUSE |
    CPDFOptimizeOption.OPTIMIZE_FLAG_BCOMPRESSIMAGE |
    CPDFOptimizeOption.OPTIMIZE_FLAG_USEFLATE;
optimizeOption.setOptimizeFlags(flags);

// Configure image compression
optimizeOption.setImageQuality(60.0f);

// Execute compression and save
```

```
// Parameter fontSubset: true means enable font subsetting
bool success = document.optimizeSaveAsCopy("output_optimized.pdf", true);

if (success)
{
    System.out.println("Document compression successful");
}
```

Comprehensive Compression Example

Below is a complete document compression example that demonstrates how to combine various optimization options:

```
public void CompressDocument(string inputPath, string outputPath)
{
    // Open document
    CPDFDocument document = new CPDFDocument();
    document.open("test.pdf");
    if (document == null)
    {
        return;
    }

    // Get original file size
    File originalFile = new File(inputPath);
    long originalSize = originalFile.length();

    // Get optimization options
    CPDFOptimizeOptions optimizeOption = document.getOptimizeOptions();
    if (optimizeOption == null)
    {
        return;
    }

    // Configure optimization flags - comprehensive optimization
    int flags =
        CPDFOptimizeOptions.OPTIMIZE_FLAG_RMNOTUSE |          // Remove unused resources
        CPDFOptimizeOptions.OPTIMIZE_FLAG_RMEPTOBJ |          // Remove empty objects
        CPDFOptimizeOptions.OPTIMIZE_FLAG_BCOMPRESSIMAGE |    // Enable image compression
        CPDFOptimizeOptions.OPTIMIZE_FLAG_RMDOCINFO |         // Remove document information
        CPDFOptimizeOptions.OPTIMIZE_FLAG_RMMEDTADATA |       // Remove metadata
        CPDFOptimizeOptions.OPTIMIZE_FLAG_USEFLATE |          // Use Flate compression
        CPDFOptimizeOptions.OPTIMIZE_FLAG_OPTIMIZEPAGECONTENT; // Optimize page content
    optimizeOption.setOptimizeFlags(flags);

    // Configure image compression options
    CPDFImageOptimizeOptions imageOption = new CPDFImageOptimizeOptions();
    imageOption.upperPpi = 225;
    imageOption.targetPpi = 150;
    imageOption.compressAlgorithm = CPDFCompressAlgorithm.JPEG2000;
    imageOption.quality = 50;
    optimizeOption.setColorImageOptions(imageOption);

    // Apply the configuration back to the document
    boolean setOk = document.setOptimizeOptions(optimizeOption);
    if (!setOk) {
        document.close();
        return;
    }

    // Perform compression
    bool success = document.optimizeSaveAsCopy(outputPath, true);

    if (success) {
        // Calculate compression ratio
        File compressedFile = new File(outputPath);
        long compressedSize = compressedFile.length();
        double compressionRatio = (1.0 - (double) compressedSize / originalSize) * 100.0;
```

```
            System.out.println("Compression completed!");
            System.out.printf("Original size: %.2f KB%n", originalSize / 1024.0);
            System.out.printf("Compressed size: %.2f KB%n", compressedSize / 1024.0);
            System.out.printf("Compression ratio: %.1f%%%n", compressionRatio);
        } else {
            System.out.println("Compression failed");
        }

        // Release resources
        document.close();
    }
```

4 Support

4.1 Reporting Problems

Thank you for your interest in ComPDFKit PDF SDK, an easy-to-use and powerful development solution to integrate high quality PDF rendering capabilities to your applications. If you encounter any technical questions or bug issues when using ComPDFKit PDF SDK for Java, please submit the problem report to the ComPDF Team. More information as follows would help us to solve your problem:

- ComPDFKit PDF SDK product and version.
- Your operating system and IDE version.
- Detailed descriptions of the problem.
- Any other related information, such as an error screenshot.

4.2 Contact Information

Website:

- Home Page: https://www.compdf.com
- Developer Guides: https://www.compdf.com/guides/pdf-sdk/java/overview
- API Reference: https://developers.compdf.com/guides/pdf-sdk/java/api-reference/index
- Code Examples: https://www.compdf.com/guides/pdf-sdk/java/examples

Contact ComPDF:

- Contact Sales: https://api.compdf.com/contact-us
- Technical Issues Feedback: https://www.compdf.com/support
- Contact Email: support@compdf.com

Thanks, The ComPDF Team